



MICROSOFT AZURE

Azure

Microsoft Cloud

100 Interview Questions

QUESTIONS & ANSWERS

From resource groups to production cloud, the 100 that matter

Azure runs a huge share of the enterprise world, and its interviews test both breadth and judgment: knowing the core services, and knowing which one to reach for and why. The questions span the fundamentals, compute and containers, storage and databases, networking and security, and the monitoring and DevOps practices that keep a cloud estate healthy. This guide covers that whole range.

Every answer explains the reasoning a strong candidate gives, with real CLI and the trade-offs behind each choice. Where it helps, it maps the Azure service to its AWS equivalent. Read it once cover to cover, then use it the night before to reload the map.

HOW TO USE THIS GUIDE

- **Open the portal or Cloud Shell.** Create a resource group and a couple of services. The hierarchy clicks fastest when you click through it.
- **Explain, do not memorize.** After each answer, say the why in your own words. That is what interviews reward.
- **Spend your last hour** on identity (Entra ID, RBAC, managed identities), networking, and storage redundancy. Those come up in almost every Azure interview.

What's inside

Part I	Fundamentals & Core — IaaS/PaaS/SaaS, regions, resource groups, ARM, subscriptions, cost	Q1–20
Part II	Compute & Containers — VMs, scale sets, App Service, Functions, AKS, ACR, Container Apps	Q21–40
Part III	Storage & Databases — Blob tiers, redundancy, Azure SQL, Cosmos DB, Redis, Data Lake	Q41–60
Part IV	Networking & Security — VNet, NSG, Load Balancer, Entra ID, RBAC, managed identities, Key Vault	Q61–80
Part V	Monitoring, DevOps & Best Practices — Azure Monitor, KQL, Azure DevOps, IaC, policy, Well-Architected	Q81–100

1 What Azure and cloud computing are

FUNDAMENTALS

Q What is Microsoft Azure, and what problem does cloud computing solve?

A Microsoft Azure is Microsoft's public cloud platform: a global network of datacenters offering on-demand compute, storage, networking, databases, and hundreds of higher-level managed services, billed on a pay-as-you-go basis. Cloud computing solves the why: instead of buying, racking, and maintaining physical hardware that you must size for peak demand and that sits idle most of the time, you rent capacity that scales up and down in minutes. You trade capital expenditure (CapEx) for operational expenditure (OpEx).

The economic model matters. On-premises you overprovision to survive spikes; in the cloud you provision for now and add more when needed, so you pay for what you consume. Elasticity means a retailer can scale out for a holiday rush and scale back afterward. This shifts undifferentiated heavy lifting (power, cooling, hardware refresh) to the provider so your team focuses on the application.

Azure spans the standard cloud characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured (metered) service. It is used for hosting web apps, running virtual machines, big-data analytics, machine learning, IoT, and integrating with on-premises Active Directory through Microsoft Entra ID.

```
az account show --output table # confirm which subscription you're signed in to
```

GOTCHA

Interviewers often ask the difference between elasticity and scalability. Scalability is the ability to add capacity; elasticity is doing it automatically in response to load, then releasing it. Cloud gives you both, but only autoscaling delivers true elasticity.

2 IaaS vs PaaS vs SaaS

FUNDAMENTALS

Q Explain IaaS, PaaS, and SaaS with Azure examples.

A These are the three cloud service models, and the why behind them is control versus responsibility. As you move from IaaS to SaaS you give up control of the underlying stack in exchange for the provider managing more of it, so your team writes less operational code. Choosing the right model is really choosing how much of the stack you want to own.

IaaS (Infrastructure as a Service) gives you virtualized hardware. You manage the OS, patching, and runtime; Azure manages the physical hosts. Example: Azure Virtual Machines and Azure Virtual Network. This is the AWS equivalent of EC2 and VPC. Use it for lift-and-shift migrations or when you need full OS control.

PaaS (Platform as a Service) gives you a managed runtime; you deploy code and Azure handles the OS, patching, and scaling. Example: Azure App Service, Azure SQL Database, Azure Functions (the AWS analogues are Elastic Beanstalk, RDS, and Lambda). **SaaS (Software as a Service)** is finished software you just consume, like Microsoft 365 or Dynamics 365 — you manage only your data and users.

```
# PaaS: deploy a web app without touching a VM
az webapp up --name myapp-123 --runtime "DOTNET:8" --sku B1
```

GOTCHA

A classic trap: "who patches the OS?" In IaaS, you do; in PaaS and SaaS, Microsoft does. Managed database offerings like Azure SQL Database are PaaS, but SQL Server running inside a VM is IaaS — the same database, a different model.

3 Regions, zones, and region pairs

GLOBAL INFRASTRUCTURE

Q What are Azure regions, availability zones, and region pairs?

A A **region** is a set of datacenters in one geographic area (for example, East US or West Europe). The why: latency and data residency. You deploy near your users to cut latency, and in a specific country to satisfy compliance and sovereignty rules. Not every service is available in every region, so region selection is an early architectural decision.

Availability zones are physically separate datacenters within a single region, each with independent power, cooling, and networking. By spreading instances across zones you survive the failure of an entire datacenter without leaving the region. Zone-redundant services replicate automatically across zones; you can also pin resources to a specific zone. This is the AWS Availability Zone concept almost one-to-one.

Region pairs are two regions in the same geography (East US pairs with West US) that Azure uses for platform-managed replication and staged updates — Microsoft never updates both halves of a pair at once. Some services like geo-redundant storage (GRS) replicate to the paired region automatically for disaster recovery. Zones protect against datacenter failure; region pairs protect against a whole region going down.

```
az vm create --resource-group rg-app --name vm1 \  
  --image Ubuntu2204 --zone 1 # pin this VM to zone 1
```

GOTCHA

Availability zones protect against a datacenter outage but not a regional disaster. For region-level resilience you need multi-region deployment or geo-redundant storage. Also, a few newer regions do not yet expose zones — verify before you design around them.

4 Resource groups

FUNDAMENTALS

Q What is an Azure resource group and how should you use it?

A A resource group is a logical container that holds related Azure resources — VMs, disks, databases, networks — for an application or workload. The why: it gives you a single boundary for lifecycle management, access control, and billing. Everything that shares a lifecycle should live together, because deleting the group deletes everything in it in one operation.

Every resource must belong to exactly one resource group. Resources can talk to resources in other groups, and you can move most resources between groups, but a resource can never span two groups. A resource group itself has a location, which is where its metadata is stored, but the resources inside it can live in different regions.

Resource groups are the natural scope for applying role-based access control (RBAC), Azure Policy, tags, and locks. A common pattern is one resource group per application per environment — for example `rg-shop-prod` and `rg-shop-dev` — so you can grant a team access to just their dev group and tear down an environment cleanly. AWS has no exact equivalent; resource groups are stronger than AWS tags-based grouping.

```
az group create --name rg-app --location eastus
az group delete --name rg-app --yes # removes every resource inside
```

GOTCHA

Deleting a resource group is irreversible and cascades to all contained resources. Put a **CanNotDelete** lock on production groups. Also, the group's own location only stores metadata — it does not force resources into that region.

5 The Azure resource hierarchy

GOVERNANCE

Q Describe the Azure management scope hierarchy from top to bottom.

A Azure organizes everything into four levels of scope: **management groups** → **subscriptions** → **resource groups** → **resources**. The why: policy and access assignments made at a higher level are inherited by everything below, so the hierarchy is how you govern hundreds of subscriptions consistently without repeating yourself.

Management groups sit above subscriptions and let you organize many subscriptions into a tree (for example, by department or by prod/non-prod). Assign a policy or RBAC role once at a management group and it flows down to every subscription beneath it. All management groups roll up to a single root management group per Microsoft Entra tenant.

Subscriptions are units of billing and management and act as a scale boundary. Resource groups hold related resources, and resources are the individual services. Because assignments inherit downward, you place broad guardrails high (management group) and specific configuration low (resource group or resource).

```
az account management-group create --name mg-corp --display-name "Corp"
az account management-group subscription add \
  --name mg-corp --subscription 00000000-0000-0000-0000-000000000000
```

GOTCHA

Inheritance is additive and generally cannot be overridden by a lower scope for a Deny effect. An interviewer may ask the order of the four scopes — memorize it top to bottom. Also, RBAC Deny and Policy Deny are different mechanisms; do not conflate them.

6 Azure Resource Manager (ARM)

CORE

Q What is Azure Resource Manager and why does it matter?

A Azure Resource Manager (ARM) is the deployment and management control plane for Azure. Every request — from the Portal, CLI, PowerShell, or an SDK — goes through the ARM REST API. The why: ARM gives you one consistent, authenticated, authorized, and audited pipeline for creating, updating, and deleting resources, no matter which tool you use.

ARM provides several cross-cutting capabilities: it applies RBAC to authorize each request, evaluates Azure Policy, records operations in the Activity Log, and lets you deploy a whole set of interdependent resources together as a template. It also understands dependencies, so it can create resources in the correct order and in parallel where possible.

Because ARM is declarative when you use templates, you describe the desired end state and ARM figures out how to reach it, making deployments repeatable and idempotent. The rough AWS equivalent is CloudFormation plus the underlying control plane. Everything you do in Azure ultimately becomes an ARM operation against a resource provider such as `Microsoft.Compute` or `Microsoft.Storage`.

```
az deployment group create --resource-group rg-app \  
  --template-file main.json --parameters @params.json
```

GOTCHA

Resource providers must be registered in a subscription before you can use them. If a deployment fails with "provider not registered," run `az provider register --namespace Microsoft.Web`. This trips up people using a new service in a fresh subscription.

7 Subscriptions and multi-subscription design

GOVERNANCE

Q What is an Azure subscription, and when would you use more than one?

A A subscription is a logical container tied to a billing account that groups resource groups and resources. It is also an administrative and scale boundary: quotas and limits (such as the number of VM cores per region) apply per subscription. The why for multiple subscriptions is isolation — of billing, of limits, and of blast radius.

You split into multiple subscriptions to separate environments (prod versus dev), to give different business units their own bill, to work around per-subscription quotas, or to enforce hard security boundaries between workloads. Each subscription trusts exactly one Microsoft Entra ID tenant for authentication, though one tenant can govern many subscriptions.

The trade-off is management overhead: more subscriptions mean more places to apply policy and RBAC, which is exactly what management groups exist to solve. A common enterprise pattern from the Cloud Adoption Framework uses separate platform, landing zone, and sandbox subscriptions. In AWS, the closest analogue is separate AWS accounts under an Organization.

```
az account list --output table
az account set --subscription "Production" # switch active subscription
```

GOTCHA

Quotas are per subscription per region, not global. Hitting a vCPU limit in one subscription does not mean you are out of capacity everywhere — you can request a quota increase or spread load across subscriptions.

8 Ways to manage Azure

TOOLING

Q What are the different ways to interact with and manage Azure?

A Azure exposes the same control plane (ARM) through several interfaces, and the why for knowing all of them is fit-for-purpose: interactive exploration versus repeatable automation call for different tools. Every tool ultimately calls the ARM REST API, so they are interchangeable in capability but not in ergonomics.

The **Azure Portal** is a browser GUI, best for learning, one-off tasks, and visualizing resources. The **Azure CLI** (**az**) is a cross-platform command-line tool using a verb-noun style, ideal for scripting on Linux and in CI/CD. **Azure PowerShell** (the **Az** module) uses cmdlets and suits Windows-centric teams and richer scripting. **Cloud Shell** is a browser-hosted shell with both preinstalled.

For programmatic control, the **SDKs** (for .NET, Python, Java, JavaScript, Go) let applications manage resources in code, and the **REST API** underlies everything. For declarative infrastructure, you use ARM templates, Bicep, or Terraform rather than imperative commands. AWS parallels: Console, AWS CLI, and language SDKs.

```
az vm list --output table           # Azure CLI
Get-AzVM | Format-Table             # Azure PowerShell equivalent
```

GOTCHA

CLI and PowerShell are equally powerful, so the choice is about your ecosystem and preference. For anything you will run more than once, prefer declarative infrastructure as code over ad hoc CLI commands so state stays reproducible.

9 ARM templates and Bicep

INFRASTRUCTURE AS CODE

Q Compare ARM templates and Bicep for infrastructure as code.

A Both are Azure's native infrastructure-as-code languages that produce declarative deployments through ARM. The why for IaC at all: describing infrastructure in version-controlled files makes deployments repeatable, reviewable, and idempotent, eliminating configuration drift and manual clicking.

ARM templates are JSON documents describing the resources you want. They are powerful but verbose, with lots of brackets, quotes, and awkward expression syntax.

Bicep is a domain-specific language that transpiles to ARM JSON — same engine, far cleaner authoring experience. Bicep offers simpler syntax, automatic dependency inference, modules, and no state file to manage because ARM tracks state on the platform side.

Bicep is now Microsoft's recommended authoring language; anything you can express in ARM JSON you can express in Bicep. The transpiled output is identical to hand-written JSON, so there is no runtime penalty. The rough AWS equivalent is CloudFormation (JSON/YAML), and Terraform is a popular third-party multi-cloud alternative that does keep its own state file.

```
// main.bicep
resource sa 'Microsoft.Storage/storageAccounts@2023-01-01' = {
  name: 'stapp${uniqueString(resourceGroup().id)}'
  location: resourceGroup().location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
}
```

GOTCHA

Bicep does not use a separate state file the way Terraform does — ARM stores deployment state. Use `az deployment group what-if` to preview changes before applying, the equivalent of a Terraform plan.

10 Tags and why they matter

GOVERNANCE

Q What are Azure tags and why are they important?

A Tags are name-value pairs you attach to resources, resource groups, and subscriptions for metadata. The why: Azure's hierarchy organizes resources one way (by group and subscription), but tags let you organize the same resources along orthogonal dimensions — cost center, environment, owner, application — that the hierarchy cannot express.

The biggest payoff is cost management: tagging every resource with a `costCenter` or `project` lets you slice the bill in Cost Management by those tags and charge back to teams. Tags also power automation (find every resource tagged `environment=dev` to shut down at night) and governance reporting.

Key limits to know: tags are not inherited by default — a resource does not automatically get its resource group's tags — though you can enforce inheritance with Azure Policy. A resource can have up to 50 tags, and some resource types do not support them. In AWS the concept is identical (resource tags), and the chargeback use case is the same.

```
az resource tag --tags environment=prod costCenter=1001 \  
  --resource-group rg-app --name vm1 --resource-type \  
  "Microsoft.Compute/virtualMachines"
```

GOTCHA

Tags do not inherit down the hierarchy automatically. If you rely on a resource group tag for cost reporting, use an Azure Policy with `modify` or `append` effect to propagate it, or resources will show up untagged.

11 The shared responsibility model

SECURITY

Q Explain the cloud shared responsibility model in Azure.

A The shared responsibility model divides security duties between Microsoft (the provider) and you (the customer). The why: in the cloud nobody owns the whole stack, so misunderstanding the boundary is how breaches happen. The line moves depending on the service model — the more managed the service, the more Microsoft owns.

Some responsibilities are always Microsoft's: the physical datacenter, physical network, and physical hosts. Some are always yours no matter the model: your data, your endpoints, your accounts and identities, and your access management. The rest — OS, network controls, applications — shifts with the model.

In **IaaS** you own the OS, patching, and application; in **PaaS** Microsoft takes the OS and runtime and you own the app and its configuration; in **SaaS** you own essentially only your data, identities, and access settings. The mnemonic: security *of* the cloud is Microsoft's, security *in* the cloud is yours. AWS describes this identically.

```
# You always own identity – enforce MFA regardless of service model
az ad user list --output table
```

GOTCHA

Data, identities, and access are your responsibility in every model, including SaaS. Assuming "the cloud handles security" leads to misconfigured storage and weak identity controls — the most common real-world cloud incidents.

12 SLAs and composite SLAs

RELIABILITY

Q What is an SLA, and how do composite SLAs work?

A A Service Level Agreement (SLA) is Microsoft's formal commitment to a service's availability, expressed as a percentage of uptime, with service credits if it is missed. The why for understanding composite SLAs: your application is only as reliable as the chain of services it depends on, and combining services almost always *lowers* the effective guarantee.

When components are in series (your app needs all of them to function), you multiply their SLAs. If a web tier at 99.95% depends on a database at 99.99%, the composite is $0.9995 \times 0.9999 \approx 99.94\%$. Every dependency you add in series can only reduce the total. This is why more moving parts means more potential downtime.

To improve reliability you add redundancy in parallel. Two independent instances each at 99.9% give a combined availability of $1 - (0.001 \times 0.001) = 99.9999\%$, because both must fail simultaneously to cause an outage. Deploying across availability zones or regions raises SLAs this way. Free tiers typically carry no financial SLA at all.

```
# Series (dependencies): multiply
# 99.95% x 99.99% = 99.94% composite
# Parallel (redundancy): 1 - (failure x failure)
```

GOTCHA

Free and Basic tiers often have no SLA. Also, an SLA is a financial promise (credits), not a guarantee of actual uptime — designing for resilience matters more than the paper number.

13 Cost management and pricing basics

COST

Q How does Azure pricing work and how do you control cost?

A Azure bills mostly on consumption — you pay for compute hours, storage GB, egress bandwidth, and per-operation charges. The why for cost management: elasticity makes it trivially easy to leave resources running or oversized, so cloud spend grows silently unless you actively monitor and govern it. Cost control is an ongoing practice, not a one-time setting.

Factors that drive price include resource type and size, region (prices vary geographically), and especially data egress — inbound data is generally free but outbound transfer across regions or to the internet is charged. Tools to plan and control spend include the Pricing Calculator and Total Cost of Ownership Calculator before deployment, and Microsoft Cost Management plus Budgets afterward.

Levers to reduce cost: right-size underused VMs, shut down non-production resources off-hours, use consumption-based serverless where load is spiky, pick cheaper storage tiers, and commit ahead with reservations or savings plans. Tagging enables chargeback and shows which teams drive spend. AWS parallels: Cost Explorer, Budgets, and the Pricing Calculator.

```
az consumption usage list --output table # review recent usage/costs
```

GOTCHA

Egress (outbound data transfer) is a frequently overlooked line item, especially cross-region traffic and data leaving Azure. Inbound is usually free; design data flows to minimize costly egress.

14 Reserved instances and savings plans

COST

Q What are reserved instances and savings plans, and when do you use each?

A Both are commitment-based discounts: you promise to spend or use a certain amount for one or three years and get a large discount (often 40–65%) versus pay-as-you-go. The why: for steady, predictable baseline workloads, paying on-demand rates leaves a lot of money on the table — you are paying a premium for flexibility you are not using.

Reserved Instances (RIs) commit to a specific resource type in a specific region — for example, a particular VM size — for one or three years. They give the deepest discount but the least flexibility; if your needs change, you can exchange or cancel with limits. Reservations exist for VMs, Azure SQL, Cosmos DB, and more.

Savings plans commit to a fixed hourly compute spend (dollars per hour) rather than a specific SKU, and automatically apply the discount to whatever eligible compute you run across regions and instance families. They trade a slightly smaller discount for much more flexibility. Rule of thumb: RIs for stable, unchanging workloads; savings plans when your compute mix shifts. AWS offers the identical pair (Reserved Instances and Savings Plans).

```
az reservations reservation-order list --output table
```

GOTCHA

Reservations and savings plans are for predictable baseline load, not spiky workloads — over-committing wastes money on capacity you do not use. Cover the steady floor with commitments and burst on pay-as-you-go.

15 Resource locks

GOVERNANCE

Q What are Azure resource locks and what problem do they solve?

A Resource locks prevent accidental deletion or modification of critical resources, independent of RBAC permissions. The why: even a user with full owner rights can fat-finger a delete on a production database. A lock is a safety catch that stops the operation regardless of how much access the person has.

There are two lock levels. `CanNotDelete` (also called Delete) lets authorized users read and modify a resource but blocks deletion. `ReadOnly` is stricter: it allows reading but blocks both modification and deletion, effectively freezing the resource. Locks can be applied at the subscription, resource group, or individual resource scope, and they inherit downward.

Because locks apply on top of RBAC, they protect against mistakes by privileged users, not against malicious actors with lock-management rights. To delete a locked resource you must first remove the lock, which forces a deliberate two-step action. Apply `CanNotDelete` to production resource groups as a standard guardrail. AWS's rough analogues are S3 Object Lock and termination protection, though Azure locks are more general.

```
az lock create --name lock-prod --lock-type CanNotDelete \  
  --resource-group rg-app
```

GOTCHA

A `ReadOnly` lock can break operations you did not expect — for example, it can prevent listing storage account keys because that is a POST operation classified as a modification. Test locks against your workflows before applying broadly.

16 Azure Advisor

OPERATIONS

Q What is Azure Advisor and what does it recommend?

A Azure Advisor is a free, personalized recommendations engine that analyzes your resource configuration and usage telemetry and suggests improvements. The why: best practices are easy to state and hard to remember to apply across a sprawling environment, so Advisor continuously scans your actual deployment and points out where you deviate.

Its recommendations map to five categories that mirror the Well-Architected Framework: **Cost** (right-size or shut down idle resources, buy reservations), **Security** (surfaced from Microsoft Defender for Cloud), **Reliability** (formerly High Availability), **Operational Excellence** (best practices and monitoring), and **Performance** (improve speed and responsiveness).

Advisor is aggregated across subscriptions and free to use — you only pay for acting on recommendations that provision resources. It is a good answer to "how do I find quick wins?" because it turns Microsoft's telemetry into a prioritized to-do list. The nearest AWS equivalent is AWS Trusted Advisor.

```
az advisor recommendation list --output table
```

GOTCHA

Advisor's security recommendations actually come from Microsoft Defender for Cloud, and some deeper security guidance requires Defender's paid plans. Advisor itself is free, but do not assume it surfaces every security finding.

17 The Well-Architected Framework

ARCHITECTURE

Q What are the five pillars of the Azure Well-Architected Framework?

A The Well-Architected Framework (WAF) is a set of guiding principles for building high-quality workloads on Azure, organized into five pillars. The why: good architecture is a series of trade-offs, and the pillars give you a shared vocabulary for making those trade-offs explicit rather than optimizing one dimension blindly.

The five pillars are: **Reliability** (the workload recovers from failure and meets its availability targets), **Security** (protect data and systems, defense in depth, least privilege), **Cost Optimization** (deliver value for the money spent, avoid waste), **Operational Excellence** (DevOps practices, monitoring, and repeatable operations), and **Performance Efficiency** (use resources efficiently and scale to meet demand).

The pillars are interdependent and often in tension — maximizing reliability with lots of redundancy raises cost, and tightening security can affect performance. The framework's value is forcing you to reason about those tensions deliberately for a given workload. AWS has a nearly identical Well-Architected Framework, with a sixth Sustainability pillar; Microsoft folds sustainability into its guidance differently.

```
# Pillars: Reliability, Security, Cost Optimization,  
#           Operational Excellence, Performance Efficiency
```

GOTCHA

The pillars trade off against each other; you cannot maximize all five at once. A strong answer names the tension — for example, redundancy improves reliability but hurts cost — and explains prioritizing based on the workload's requirements.

18 Cloud Adoption Framework and landing zones

ARCHITECTURE

Q What is the Cloud Adoption Framework, and what is a landing zone?

A The Cloud Adoption Framework (CAF) is Microsoft's end-to-end guidance for an organization's entire cloud journey — the business and process side, not just architecture. The why: most cloud failures are organizational, not technical, so CAF provides a structured methodology (Strategy, Plan, Ready, Adopt, Govern, Manage) to align business goals with technical execution.

A **landing zone** is the output of the Ready phase: a preconfigured, secure, scalable environment that follows best practices and is ready to host workloads from day one. Rather than each team building networking, identity, and governance from scratch, the platform team provisions a landing zone with those foundations baked in, and application teams deploy into it.

Landing zones typically separate **platform** subscriptions (shared identity, management, and connectivity) from **application** landing zones where workloads run, all governed by management groups, policy, and RBAC. This is enterprise-scale governance made repeatable. The AWS parallels are the Cloud Adoption Framework and AWS Control Tower / Landing Zone.

```
# Landing zones are commonly deployed via Bicep or Terraform
az deployment mg create --management-group-id mg-corp \
  --location eastus --template-file landingzone.bicep
```

GOTCHA

Do not confuse CAF with the Well-Architected Framework. CAF is about the organization's journey and governance (people and process); WAF is about the quality of an individual workload's architecture. Interviewers test whether you can distinguish them.

19 Azure vs AWS service mapping

COMPARISON

Q Map the major Azure services to their AWS equivalents.

A Azure and AWS offer broadly the same building blocks under different names, so knowing the mapping helps you translate architectures across clouds. The why for the question: it tests whether you understand what a service *does* rather than just its brand name — the concepts transfer even when the naming does not.

Core compute and storage: Azure Virtual Machines ↔ EC2; Azure Functions ↔ Lambda; App Service ↔ Elastic Beanstalk; Blob Storage ↔ S3; Azure Kubernetes Service (AKS) ↔ EKS. Databases: Azure SQL Database ↔ RDS; Cosmos DB ↔ DynamoDB.

Networking and identity: Virtual Network ↔ VPC; Azure Load Balancer ↔ ELB/NLB; Azure DNS ↔ Route 53; Microsoft Entra ID ↔ IAM plus AWS Directory. Management: ARM/Bicep ↔ CloudFormation; Azure Monitor ↔ CloudWatch; Cost Management ↔ Cost Explorer. The mappings are approximate — feature sets and boundaries differ — but the categories line up.

```
# VMs=EC2  Functions=Lambda  Blob=S3  AKS=EKS
# SQL DB=RDS  Cosmos DB=DynamoDB  VNet=VPC  Entra ID=IAM
```

GOTCHA

The mappings are not one-to-one. For example, Microsoft Entra ID is an identity provider (closer to AWS IAM plus Cognito plus Directory Service combined), and Cosmos DB is multi-model, unlike DynamoDB. Note the differences, not just the analogy.

20 Azure Arc

HYBRID

Q What is Azure Arc and what problem does it solve?

A Azure Arc extends Azure's management plane (ARM) to resources that live outside Azure — on-premises servers, other clouds like AWS and GCP, and edge locations. The why: enterprises rarely run everything in one cloud, and managing each environment with its own tools is fragmented. Arc projects external resources into Azure so you govern them with the same single control plane.

Once a server or Kubernetes cluster is Arc-enabled, it appears in Azure as a resource. You can then apply **Azure Policy**, **RBAC**, **tags**, **Azure Monitor**, **Microsoft Defender for Cloud**, and even run certain Azure data services (like Azure SQL Managed Instance) on that hardware — all managed as if it were native Azure.

Arc supports several resource types: Arc-enabled servers, Arc-enabled Kubernetes, and Arc-enabled data services. The value is one consistent governance, security, and inventory story across a hybrid and multi-cloud estate. AWS's nearest counterparts are EKS Anywhere and Systems Manager, but Arc's ambition to bring the full Azure control plane everywhere is broader.

```
az connectedmachine list --output table # view Arc-enabled servers
az connectedk8s connect --name my-onprem-cluster --resource-group rg-arc
```

GOTCHA

Arc manages and governs external resources through Azure's control plane, but the workloads still run on your hardware — Arc does not move them into Azure. It is about consistent management, not migration.

21 Azure Virtual Machines

COMPUTE

Q What is an Azure Virtual Machine, and what are you responsible for when you run one?

A An Azure Virtual Machine is an on-demand, scalable compute resource that gives you a full operating system running on Microsoft-managed hypervisor hardware. It is the classic Infrastructure-as-a-Service (IaaS) building block: Azure runs the physical host, networking fabric, and hypervisor, while you own everything from the OS upward.

The WHY matters here. The shared-responsibility model shifts with each service tier. With a VM you patch the OS, install runtimes, configure firewalls, harden the image, and manage backups. This maximum control is exactly why you would choose a VM over a PaaS option: legacy apps, custom kernel modules, specific licensing, or lift-and-shift migrations where re-architecting is not worth it.

A VM is composed of several separable resources: the VM object itself, one or more managed disks (OS + data), a NIC attached to a subnet, an optional public IP, and a network security group controlling traffic. Because these are decoupled, you can detach a disk and reattach it to another VM, or delete the VM while keeping the disk.

Sizing is chosen from families: B-series (burstable, cheap), D-series (general purpose), E-series (memory optimized), F-series (compute optimized), and N-series (GPU). Picking the wrong family is the most common cost mistake.

```
az vm create -g rg -n web01 \  
  --image Ubuntu2204 --size Standard_D2s_v5 \  
  --admin-username azureuser --generate-ssh-keys
```

GOTCHA

Stopping a VM in the portal (guest OS shutdown) still bills you. To stop billing for compute you must **deallocate** it, which releases the underlying host. A "Stopped" state and a "Stopped (deallocated)" state are not the same thing.

22 VM Scale Sets

COMPUTE

Q What problem do Virtual Machine Scale Sets solve, and how do they differ from a plain VM?

A A Virtual Machine Scale Set (VMSS) lets you deploy and manage a group of identical, load-balanced VMs as a single resource that can automatically grow and shrink. Instead of hand-creating ten web servers, you define one model and a capacity, and the scale set instantiates and maintains them.

The WHY is elasticity plus uniformity. Because every instance is built from the same image and configuration, you get consistent behavior and can roll updates across the fleet in batches. Combined with autoscale rules driven by CPU, memory, or custom metrics, the set adds instances during load spikes and removes them when demand falls, so you pay for capacity you actually use.

VMSS supports two orchestration modes. **Uniform** mode treats instances as fungible and is optimized for large stateless fleets. **Flexible** mode gives you the elasticity of a scale set while letting instances behave more like individually managed VMs, spread across availability zones and fault domains. Flexible is now the recommended default for most scenarios.

```
az vmss create -g rg -n webset \  
  --image Ubuntu2204 --instance-count 3 \  
  --orchestration-mode Flexible --upgrade-policy-mode automatic
```

GOTCHA

Scale sets are ideal for stateless workloads. If your app stores session or data on the local disk, scaling in will destroy instances and lose that state. Externalize state to a database, cache, or blob storage first.

COMPUTE

Q What is the difference between an availability set and an availability zone, and which SLA does each give you?

A Both protect VM workloads from failure, but at different blast radii. An availability set protects you inside a single datacenter; an availability zone protects you across physically separate datacenters within a region.

An **availability set** distributes VMs across fault domains (separate racks with independent power and network) and update domains (grouped so Azure never reboots them all at once during host maintenance). This guards against rack-level hardware failure and planned maintenance, and yields a 99.95% VM SLA. But if the whole datacenter loses power, every VM in the set is down.

An **availability zone** is a physically isolated location within a region, each with its own power, cooling, and networking. Spreading VMs across two or more zones protects against an entire datacenter failing and delivers a 99.99% SLA. The WHY of choosing zones is disaster tolerance at regional scale; the tradeoff is potential inter-zone latency and cross-zone data transfer charges.

A key rule: you cannot change a running VM from an availability set to a zone, or move it between zones, without redeploying it.

```
# Zone-pinned VM
az vm create -g rg -n vm1 --image Ubuntu2204 --zone 1
# Availability-set VM
az vm create -g rg -n vm2 --availability-set myAvSet
```

GOTCHA

Not every region supports availability zones, and not every VM size is available in every zone. Always confirm zone support before designing around it, or your deployment will fail at scale-out time.

24 Azure App Service

COMPUTE

Q What is Azure App Service and why choose it over running your own web server on a VM?

A Azure App Service is a fully managed Platform-as-a-Service for hosting web apps, REST APIs, and mobile backends. You deploy code or a container and Azure handles the OS, patching, load balancing, and the underlying VMs entirely.

The WHY is that most web workloads do not benefit from OS-level control; they benefit from fast deployment, built-in scaling, and zero patching overhead. App Service gives you HTTPS with managed certificates, custom domains, CI/CD integration, deployment slots, auth/authorization (Easy Auth), and autoscale, all without you managing a single server. You trade fine-grained control for dramatically less operational burden.

It supports .NET, Java, Node.js, Python, PHP, and Ruby natively, plus custom containers for anything else. Apps run inside an App Service plan, which is the actual compute (the VMs) that hosts one or more apps.

For network isolation or compliance, an **App Service Environment (ASE)** gives you a single-tenant, VNet-injected deployment of App Service, at a premium price.

```
az webapp up -n myapp -g rg \  
  --runtime "NODE:20-lts" --sku P1v3
```

GOTCHA

By default outbound App Service traffic is not in your VNet. To reach private resources (a database with no public endpoint) you must enable **VNet Integration**, and to control inbound access use Private Endpoints or access restrictions.

25 App Service Plans and Pricing Tiers

COMPUTE

Q How do App Service plans and their pricing tiers work, and what do you get at each tier?

A An App Service plan defines the region, the VM size, the instance count, and the pricing tier that a set of apps runs on. Crucially, all apps in the same plan share that compute, so ten small apps in one plan compete for the same CPU and memory.

The WHY of understanding tiers is cost-versus-capability tradeoff. Tiers ladder up in features:

Free/Shared (F1, D1) run on shared infrastructure with strict quotas, no custom domains on Free, and no SLA, meant for dev/test only. **Basic (B1-B3)** gives dedicated VMs, custom domains, and TLS but no autoscale or slots. **Standard (S1-S3)** adds autoscale, up to 5 deployment slots, and daily backups. **Premium (P1v3-P3v3)** adds more power, more slots, VNet integration, and better scale limits. **Isolated (I1v2+)** is the App Service Environment tier for single-tenant, VNet-native hosting.

Billing is per plan, not per app, and you pay whether or not apps are running (except Free/Shared and the separate Consumption model used by Functions).

```
az appservice plan create -g rg -n myplan \
  --sku S1 --number-of-workers 2
```

GOTCHA

Because plan cost is fixed regardless of app count, consolidating many low-traffic apps into one plan saves money, but a single noisy app can starve the others. Isolate performance-sensitive apps in their own plan.

26 Azure Functions

SERVERLESS

Q What are Azure Functions and what does "serverless" actually mean here?

A Azure Functions is an event-driven, serverless compute service that runs small pieces of code (functions) in response to triggers, without you provisioning or managing any servers. "Serverless" does not mean there are no servers; it means the servers are invisible to you and you are billed for execution, not for idle capacity.

The WHY is a programming model built around events. A function has exactly one **trigger** (HTTP request, queue message, blob upload, timer, Event Grid event, Cosmos DB change feed) and optional **bindings** that declaratively connect inputs and outputs to services. Bindings eliminate boilerplate: instead of writing SDK code to read a queue and write to a table, you declare them and Azure wires up the plumbing.

On the Consumption plan, functions scale automatically from zero to many instances based on incoming events, and you pay only for execution time and memory (GB-seconds) plus number of executions. This makes them ideal for spiky, intermittent, or glue workloads.

```
# Timer-triggered function (function.json / decorator)
az functionapp create -g rg -n myfuncs \
  --consumption-plan-location eastus \
  --runtime python --functions-version 4 \
  --storage-account mystorage
```

GOTCHA

Functions on Consumption have an execution timeout (default 5 min, max 10 min). Long-running work must be broken up, moved to Durable Functions, or hosted on a Premium/Dedicated plan.

27 Consumption vs Premium vs Dedicated Function Plans

SERVERLESS

Q Compare the Consumption, Premium, and Dedicated hosting plans for Azure Functions.

A These three plans trade off cost, cold-start behavior, and control. Choosing correctly is mostly about your latency tolerance and traffic shape.

Consumption is pure pay-per-execution. It scales to zero when idle (so you pay nothing) and scales out automatically. The cost is cold starts, since instances spin up on demand, and a hard execution timeout. Best for spiky, infrequent workloads where occasional latency is fine.

Premium (Elastic Premium, EP) keeps pre-warmed instances always ready, eliminating cold starts, and adds VNet integration, longer/unbounded timeouts, and more powerful instances. You pay for at least one always-ready instance, so it is never free, but it scales elastically above that baseline. Best for latency-sensitive or VNet-bound production APIs.

Dedicated (App Service plan) runs functions on the same VMs as your web apps at a fixed cost. It makes sense when you already have an underutilized App Service plan, need predictable pricing, or require features only available there. It does not scale to zero.

```
az functionapp plan create -g rg -n prempplan \  
  --sku EP1 --min-instances 1 --max-burst 20 \  
  --location eastus
```

GOTCHA

VNet integration for Functions requires Premium or Dedicated. If your function must reach a private database, Consumption will not work, a common surprise late in a project.

28 Durable Functions

SERVERLESS

Q What are Durable Functions and what stateful patterns do they enable?

A Durable Functions is an extension of Azure Functions that lets you write stateful, long-running workflows in code while remaining serverless. Normal functions are stateless; Durable Functions add an orchestration layer that transparently checkpoints progress so a workflow can span minutes, hours, or days.

The WHY is that many real problems are inherently stateful: multi-step approval flows, fan-out/fan-in parallel processing, or waiting on a human. Durable Functions solve this with three roles. An **orchestrator function** defines the workflow logic and calls activities; it must be deterministic because it replays history to rebuild state. **Activity functions** do the actual work (the side effects). An **entity function** models durable stateful objects (the actor pattern).

It supports named patterns: function chaining (A then B then C), fan-out/fan-in (run many activities in parallel then aggregate), async HTTP APIs with status polling, monitoring, and human interaction with timeouts. State is persisted automatically in a storage backend.

```
# Orchestrator (C#) - replay-safe, no I/O directly
var outputs = new List<string>();
outputs.Add(await ctx.CallActivityAsync<string>("SayHello", "Tokyo"));
outputs.Add(await ctx.CallActivityAsync<string>("SayHello", "Seattle"));
```

GOTCHA

Orchestrator code must be deterministic: no `DateTime.Now`, no random numbers, no direct I/O, no non-durable async calls. Because the runtime replays the orchestrator to rebuild state, non-deterministic code produces corrupt or inconsistent results.

29 Cold Starts

SERVERLESS

Q What is a cold start, why does it happen, and how do you mitigate it?

A A cold start is the extra latency incurred when a serverless platform must allocate a new instance, load the runtime, and initialize your app before it can handle a request. It happens because Consumption-plan Functions (and similar services) scale to zero when idle, so the first request after a quiet period pays the full startup cost.

The WHY of the delay: the platform must provision a worker, mount your code, start the language runtime (a heavy .NET or JVM startup is slower than Node or Python), run your initialization code, and establish connections. This can range from sub-second to several seconds.

Mitigations, roughly in order of effectiveness: move to the **Premium plan** with always-ready/pre-warmed instances, which essentially eliminates cold starts; keep the app small and dependencies lean; avoid heavy work in the global/startup path; use languages with faster startup; and for HTTP apps consider a health-check "keep-warm" timer trigger to prevent scale-to-zero (a workaround, not a true fix).

```
# Keep-warm timer: fires every 5 minutes
schedule: "0 */5 * * * *"
# Or eliminate entirely with Premium always-ready
az functionapp plan update -g rg -n preplan --min-instances 1
```

GOTCHA

A keep-warm timer keeps one instance hot but does nothing for cold starts during scale-out, when a traffic spike forces new instances. Only Premium pre-warmed instances address that case.

30 Azure Container Instances (ACI)

CONTAINERS

Q What is Azure Container Instances and when would you use it instead of AKS?

A Azure Container Instances (ACI) is the simplest way to run a container in Azure: you give it an image and it runs, with no VMs to manage and no orchestrator to operate. It is serverless containers, billed per second based on the vCPU and memory you request.

The WHY is that not every containerized workload needs Kubernetes. ACI shines for short-lived, bursty, or simple scenarios: batch jobs, build agents, event-driven processing, or quick isolated tasks. Startup is fast (seconds), and because there is no cluster, there is no per-node overhead or control-plane management.

You can group related containers into a **container group** that shares a lifecycle, network, and storage, similar to a Kubernetes pod. ACI supports mounting Azure Files, restart policies, environment variables/secrets, and VNet deployment. What it lacks is orchestration: no service discovery, no rolling updates, no autoscaling of replicas, no self-healing beyond restart policy.

A common pattern is the Virtual Kubelet, which lets AKS burst overflow pods onto ACI so you get elastic capacity without adding nodes.

```
az container create -g rg -n job1 \  
  --image mcr.microsoft.com/azuredocs/aci-helloworld \  
  --cpu 1 --memory 1.5 --restart-policy Never
```

GOTCHA

ACI is not built for long-running, always-on services that need scaling and self-healing. Reach for Container Apps or AKS there. For genuinely bursty or one-shot jobs, ACI's per-second billing is far cheaper than idling a cluster.

31 Azure Kubernetes Service (AKS)

CONTAINERS

Q What is Azure Kubernetes Service and what does Azure manage versus what do you manage?

A Azure Kubernetes Service (AKS) is a managed Kubernetes offering where Azure operates the Kubernetes control plane for you (free), and you manage the worker node pools that run your workloads. It removes most of the operational pain of running Kubernetes yourself while keeping the full Kubernetes API.

The WHY of the split: the control plane (API server, scheduler, etcd, controller manager) is complex and critical, so Azure hosts, patches, and scales it, and offers an optional uptime SLA. You are responsible for the **node pools** (the VMs), meaning you choose sizes, apply node upgrades, and pay for those VMs. Your pods, deployments, services, and ingress are all standard Kubernetes objects you manage with `kubectl`.

AKS integrates deeply with Azure: Microsoft Entra ID for RBAC, Azure CNI for VNet-native pod networking, managed identities, Azure Monitor for containers, and ACR for images. You can have multiple node pools, for example a system pool plus GPU or spot pools.

```
az aks create -g rg -n myaks --node-count 3 \
  --enable-managed-identity --generate-ssh-keys \
  --attach-acr myregistry
az aks get-credentials -g rg -n myaks
```

GOTCHA

The control plane is free, but you pay for every node VM, load balancer, and disk. An idle AKS cluster still costs money, unlike ACI or Consumption Functions. Right-size node pools and use the cluster autoscaler.

32 AKS Scaling

CONTAINERS

Q Explain the three layers of scaling in AKS: cluster autoscaler, HPA, and KEDA.

A AKS scales at two distinct levels, and it is essential to know which lever does what. Pod-level scaling adds more replicas of your app; node-level scaling adds more VMs to host those replicas. They work together but solve different problems.

The **Horizontal Pod Autoscaler (HPA)** is a Kubernetes controller that increases or decreases the number of pod replicas based on observed metrics, classically CPU and memory. It answers "do I have enough copies of my app running?" but it cannot create capacity that does not exist.

The **Cluster Autoscaler** operates at the node level. When pods are stuck **Pending** because no node has room, it adds nodes to the pool; when nodes are underused, it removes them. It answers "do I have enough VMs to place my pods?" HPA and Cluster Autoscaler complement each other: HPA adds pods, and if they cannot fit, Cluster Autoscaler adds nodes.

KEDA (Kubernetes Event-Driven Autoscaling) extends HPA to scale on external event sources, like queue length, Kafka lag, or Service Bus depth, and can scale to zero. It is the right tool for event-driven workloads where CPU is a poor proxy for load.

```
kubectl autoscale deployment web --cpu-percent=70 --min=2 --max=10
az aks update -g rg -n myaks --enable-cluster-autoscaler \
  --min-count 3 --max-count 10
```

GOTCHA

HPA needs accurate resource **requests** on your pods to compute utilization. If you omit requests, CPU-based HPA silently does nothing because it has no baseline to measure against.

33 Azure Container Registry (ACR)

CONTAINERS

Q What is Azure Container Registry and how do you securely connect it to AKS?

A Azure Container Registry (ACR) is a managed, private Docker/OCI registry for storing and distributing container images and related artifacts (Helm charts, OCI artifacts). It keeps your images close to your compute, geo-replicable, and access-controlled by Azure identity rather than static credentials.

The WHY of a private registry is security and locality. Instead of pulling from public Docker Hub (rate limits, supply-chain risk), you push vetted images to ACR and control who can pull them via Entra ID and RBAC. ACR also offers image scanning (via Defender), content trust, ACR Tasks for cloud-based builds and automated rebuilds on base-image updates, and geo-replication for multi-region pulls.

It comes in three tiers: **Basic**, **Standard**, and **Premium**. Only Premium supports geo-replication, private endpoints, and content trust, and it offers the highest throughput and storage.

The secure way to let AKS pull from ACR is to attach the registry so AKS's managed identity is granted the **AcrPull** role, avoiding image-pull secrets entirely.

```
az acr create -g rg -n myregistry --sku Premium
az acr build -r myregistry -t app:v1 .
az aks update -g rg -n myaks --attach-acr myregistry
```

GOTCHA

Don't distribute registry admin credentials or image-pull secrets to clusters. Attaching ACR (granting **AcrPull** to the kubelet identity) is the recommended, credential-free approach and is far easier to rotate and audit.

34 Azure Container Apps

CONTAINERS

Q What is Azure Container Apps and where does it sit between ACI and AKS?

A Azure Container Apps (ACA) is a serverless container platform built on Kubernetes, KEDA, Dapr, and Envoy, but it hides all of that so you never touch the Kubernetes API. It is the sweet spot for teams that want container orchestration features without operating a cluster.

The WHY is that ACI is too bare (no scaling, no ingress, no revisions) while AKS is too much (you run the cluster). Container Apps gives you the orchestration essentials out of the box: automatic HTTP and event-driven autoscaling including **scale to zero** via built-in KEDA, HTTPS ingress with traffic splitting, blue-green **revisions**, service discovery, and optional Dapr integration for microservice patterns like pub/sub and state.

Billing is consumption-based, so a scaled-to-zero app costs nothing for compute. This makes ACA excellent for microservices, HTTP APIs, background processors, and event-driven jobs where you want elasticity without cluster management.

You do give up direct Kubernetes control: no `kubectl`, no custom operators, no arbitrary Helm charts. If you need those, AKS is the answer.

```
az containerapp create -g rg -n api \  
  --environment myenv --image myregistry.azurecr.io/app:v1 \  
  --ingress external --target-port 8080 \  
  --min-replicas 0 --max-replicas 10
```

GOTCHA

Scale-to-zero means the first request after idle incurs a cold start. For latency-sensitive endpoints set `--min-replicas 1` to keep one instance warm, trading a little cost for consistent response times.

35 Deployment Slots

COMPUTE

Q What are App Service deployment slots and how do they enable zero-downtime releases?

A Deployment slots are live, addressable instances of your web app running in the same App Service plan, each with its own hostname. The default slot is production; you create additional slots like `staging` to deploy and validate a new version before it goes live.

The WHY is zero-downtime, low-risk releases via **slot swapping**. You deploy the new build to staging, warm it up, run smoke tests against its real URL, then swap staging with production. The swap is a routing switch, not a redeploy, so it is near-instant and, critically, reversible: if something breaks you swap back to instantly roll back to the previous version.

Before completing a swap, Azure performs a warm-up by sending requests to the new instance so it is ready before it receives production traffic, avoiding a cold-start hit at go-live. You can also route a percentage of traffic to a slot for canary testing.

App settings and connection strings can be marked as **slot-specific ("sticky")** so they stay with the slot and do not swap.

```
az webapp deployment slot create -g rg -n myapp --slot staging
# deploy to staging, test, then swap
az webapp deployment slot swap -g rg -n myapp \
  --slot staging --target-slot production
```

GOTCHA

Slots require Standard tier or higher, and every slot runs on the same plan, consuming its resources. Also, forgetting to mark environment-specific settings as sticky means a swap can push staging config (or a test database connection) into production.

36 Autoscaling

COMPUTE

Q How does Azure autoscaling work, and what is the difference between metric-based and schedule-based rules?

A Azure autoscale (via Azure Monitor autoscale) automatically adjusts the number of instances for resources like VM Scale Sets and App Service plans according to rules you define. The goal is to match capacity to demand, so you neither overpay for idle instances nor degrade under load.

Metric-based (reactive) scaling triggers on observed telemetry: CPU percentage, memory, queue length, request count, or custom metrics. You define scale-out and scale-in rules, for example "add 2 instances when average CPU > 70% for 10 minutes" and "remove 1 when CPU < 30%." The WHY of reacting to metrics is handling unpredictable load, but it lags because it must observe the load before responding.

Schedule-based (proactive) scaling changes capacity on a calendar: scale up to 10 instances at 8 AM on weekdays, back down to 2 at night. This is ideal for predictable patterns like business hours, because it provisions capacity before demand arrives rather than after.

The two combine well: a schedule sets a sensible baseline (higher minimum during business hours), and metric rules handle surprises on top. Always define both a scale-out and a matching scale-in rule, plus min/max bounds.

```
az monitor autoscale create -g rg --resource myset \  
  --resource-type Microsoft.Compute/virtualMachineScaleSets \  
  --min-count 2 --max-count 10 --count 3  
az monitor autoscale rule create -g rg --autoscale-name myset \  
  --condition "Percentage CPU > 70 avg 10m" --scale out 2
```

GOTCHA

Set the scale-in threshold well below scale-out and use cool-down periods, or you get *flapping*: instances added and removed repeatedly around a boundary, causing churn and instability instead of smooth capacity.

37 Spot VMs

COMPUTE

Q What are Azure Spot VMs, how much can they save, and what is the catch?

A Azure Spot Virtual Machines let you buy Azure's unused compute capacity at deep discounts, often up to 90% off pay-as-you-go, in exchange for the risk that Azure can reclaim ("evict") them at any time when it needs the capacity back or when the price exceeds your cap.

The WHY of the discount is that Spot capacity is genuinely spare. There is no availability guarantee and no SLA, so you must design around eviction. When Azure evicts, it gives a short (30-second) notice via Scheduled Events, and you choose the eviction policy: **Deallocate** (stop and keep the disk so you can restart later) or **Delete** (remove entirely).

This makes Spot ideal for interruption-tolerant, stateless, or checkpointable workloads: batch processing, big-data jobs, CI/CD build agents, rendering, dev/test, and AKS spot node pools for fault-tolerant pods. It is unsuitable for stateful databases or anything needing guaranteed uptime.

You can set a maximum price you are willing to pay; if the spot price rises above it, the VM is evicted.

```
az vm create -g rg -n batch01 --image Ubuntu2204 \
  --priority Spot --eviction-policy Deallocate \
  --max-price -1 # -1 = pay up to on-demand price
```

GOTCHA

Setting `--max-price -1` means you never get evicted for price (only for capacity), but you also lose the price-cap protection, so a cost spike bills at the full on-demand rate. Choose the cap deliberately.

38 Choosing VM vs App Service vs Functions vs AKS

COMPUTE

Q How do you decide between a VM, App Service, Azure Functions, and AKS for a workload?

A The decision is fundamentally about how much control you need versus how much operational burden you want to avoid, plus the shape of your workload. Moving from VM to Functions trades control for convenience.

Choose a **Virtual Machine** when you need full OS control: legacy apps, custom software, specific kernels, licensing, or lift-and-shift where re-architecting is not worth it. You accept patching and management in return for maximum flexibility.

Choose **App Service** for standard web apps and APIs where you want fast deployment, managed TLS, slots, and autoscale without managing servers. It is the default for typical web workloads.

Choose **Azure Functions** for event-driven, short-lived, or bursty logic, glue code, scheduled jobs, and lightweight APIs, where scale-to-zero and pay-per-execution shine. Choose **AKS** when you have many containerized microservices needing orchestration, service mesh, complex networking, or portability across clouds, and you have the team to operate Kubernetes. Consider Container Apps as the middle ground: containers with orchestration but no cluster to run.

```
# Rule of thumb ladder (least to most ops burden):  
# Functions < App Service / Container Apps < AKS < VM
```

GOTCHA

Do not default to AKS because it is powerful. Kubernetes carries real operational cost. If you have a handful of services, App Service or Container Apps will deliver faster with far less overhead.

39 Azure Batch

COMPUTE

Q What is Azure Batch and what kind of workloads is it designed for?

A Azure Batch is a managed service for running large-scale parallel and high-performance computing (HPC) batch jobs across a pool of VMs. You define a pool of compute nodes and submit jobs made of many tasks, and Batch handles scheduling, distributing tasks to nodes, retrying failures, and scaling the pool.

The WHY is that certain workloads are embarrassingly parallel: rendering thousands of animation frames, running Monte Carlo simulations, transcoding media, genomic analysis, or financial risk modeling. Writing your own scheduler and node-management layer for these is painful; Batch provides it as a service. It handles queuing tasks, placing them on available nodes, collecting results, and automatically scaling nodes based on the workload via formulas.

Batch pools can use regular or **low-priority/Spot** VMs to slash cost for interruption-tolerant work, and can run Linux or Windows nodes with custom images and applications. It is a natural fit when you have a queue of independent tasks and want to fan them out across a fleet, then tear the fleet down when done.

```
az batch pool create --id renderpool \  
  --vm-size Standard_D4s_v5 --target-dedicated-nodes 10 \  
  --image canonical:0001-com-ubuntu-server-jammy:22_04-lts
```

GOTCHA

Batch is for finite, task-based, parallel jobs, not always-on services or complex inter-service orchestration. For long-running microservices use AKS or Container Apps; for stateful multi-step workflows use Durable Functions or Logic Apps.

40 Managed Identity for Compute

SECURITY

Q What is a managed identity and how does it let compute access other Azure services securely?

A A managed identity is an automatically managed service principal in Microsoft Entra ID that an Azure compute resource (VM, App Service, Function, Container App, AKS) can use to authenticate to other Azure services without any credentials stored in code or config. It is the answer to "how do I connect to Key Vault or SQL without a password in my app?"

The WHY is eliminating secret sprawl. Traditionally you would store a connection string or client secret somewhere, and secrets leak, expire, and must be rotated. A managed identity has no secret you ever see: Azure provisions and rotates the underlying credential, and your code just requests a token from the local identity endpoint, which downstream services validate via Entra ID and RBAC.

There are two types. A **system-assigned** identity is tied to a single resource's lifecycle and deleted with it, one-to-one. A **user-assigned** identity is a standalone resource you create once and attach to many resources, ideal for sharing an identity across a fleet or pre-provisioning access. In both cases you grant the identity RBAC roles (for example **Key Vault Secrets User**) on the target resource.

```
az webapp identity assign -g rg -n myapp
# grant it access to a key vault
az role assignment create --assignee <principalId> \
  --role "Key Vault Secrets User" --scope <vaultId>
```

GOTCHA

Assigning a managed identity does nothing on its own; you must also grant it RBAC roles (or a Key Vault access policy) on each target resource. A common failure is enabling the identity but forgetting the role assignment, then getting 403s at runtime.

41 Storage accounts overview

STORAGE

Q What is an Azure Storage account and what does it contain?

A A storage account is the top-level namespace that groups together Azure's core data services: Blob, File, Queue, and Table storage. The reason it exists is billing and management: it is the unit at which you configure redundancy, performance tier, access keys, networking rules, and encryption. Everything inside it shares those settings and a single pair of access keys.

The account name must be globally unique because it forms the DNS endpoint, for example `mystore.blob.core.windows.net`. Each service gets its own endpoint suffix (**blob**, **file**, **queue**, **table**, **dfs** for Data Lake).

The dominant type today is **StorageV2 (general-purpose v2)**, which supports all services and all tiers. Specialized kinds exist: **BlobStorage**, **BlockBlobStorage** (premium block blobs), and **FileStorage** (premium files). Performance is either Standard (HDD-backed) or Premium (SSD-backed, lower latency).

```
az storage account create -n mystore -g rg \
  --sku Standard_LRS --kind StorageV2 --location eastus
```

GOTCHA

Access keys grant full control over the entire account, so treat them like root credentials. Prefer Azure AD (Entra ID) RBAC or SAS tokens, and rotate the two keys on a schedule so you can swap without downtime.

42 Blob access tiers

STORAGE

Q Explain the blob access tiers: hot, cool, cold, and archive.

A The tiers trade storage cost against access cost and latency, based on how often you read the data. The core idea: the colder the tier, the cheaper it is to store bytes, but the more expensive and slower it is to read them. You pick the tier that matches your access pattern to minimize total cost.

Hot is for frequently accessed data: highest storage price, lowest access price. **Cool** targets infrequent access with a 30-day minimum retention. **Cold** is even cheaper storage with a 90-day minimum, still online. **Archive** is the cheapest storage but offline: data must be rehydrated (minutes to hours) before you can read it, and it has a 180-day minimum.

Hot, cool, and cold can be set at the account default level or per-blob. Archive is per-blob only. Deleting or moving a blob before its minimum retention triggers an early-deletion charge.

```
az storage blob set-tier --account-name mystore \  
-c logs -n old.log --tier Archive
```

GOTCHA

Rehydrating from Archive is not instant. Standard-priority rehydration can take up to 15 hours. If you might need the data quickly, don't archive it; use cold instead, which stays online.

43 Storage redundancy

STORAGE

Q Compare LRS, ZRS, GRS, RA-GRS, and GZRS.

A Redundancy determines how many copies of your data exist and where, protecting against hardware, datacenter, or regional failures. Azure always keeps at least three copies; the options differ in whether those copies span availability zones or regions. More spread means higher durability but higher cost and, for geo options, some replication lag.

LRS keeps three copies within a single datacenter: cheapest, survives disk or node failure but not a datacenter loss. **ZRS** spreads three copies across three availability zones in the region, surviving a full zone outage. **GRS** is LRS plus asynchronous replication to a paired secondary region hundreds of miles away. **GZRS** combines ZRS in the primary with geo-replication to the secondary: the strongest option.

By default the geo-secondary is not readable; you only reach it after a failover. The **RA-** (read-access) variants, **RA-GRS** and **RA-GZRS**, expose a read-only endpoint at the secondary so apps can read even before failover.

```
az storage account update -n mystore -g rg \  
  --sku Standard_RAGZRS
```

GOTCHA

Geo-replication is asynchronous, so a sudden regional loss can lose the last few seconds to minutes of writes (measured by the RPO). GRS is not a substitute for backups; it will happily replicate a bad delete to the secondary.

44 Azure Files

STORAGE

Q What is Azure Files and when would you use it over Blob storage?

A Azure Files provides fully managed file shares accessible over the industry-standard SMB and NFS protocols. The key difference from Blob storage is the access model: Files presents a real mountable filesystem with directories and file locking, so existing applications that expect a network drive work unchanged, whereas Blob requires REST or SDK calls.

You mount a share as a network drive on Windows, Linux, or macOS, or from Azure VMs and containers. This makes it ideal for lift-and-shift migrations, shared configuration, and replacing on-premises file servers. Azure File Sync extends this by caching shares on local Windows servers while keeping the cloud copy as the master.

Choose Files when you need shared filesystem semantics or POSIX/SMB compatibility; choose Blob for massive unstructured object storage, static web content, or data-lake workloads where object APIs and lifecycle rules matter more.

```
# mount an SMB share on Linux
sudo mount -t cifs //mystore.file.core.windows.net/data /mnt/data \
-o vers=3.0,username=mystore,password=$KEY,serverino
```

GOTCHA

Many ISPs block outbound port 445 (SMB), which breaks direct mounts from home or office networks. Use a VPN, ExpressRoute, or private endpoint to reach the share securely.

45 Queue storage

STORAGE

Q What is Azure Queue storage and how does it differ from Service Bus queues?

A Queue storage is a simple, massively scalable message queue for decoupling application components. A producer enqueues messages and a consumer dequeues and processes them, which lets you smooth out load spikes and build reliable asynchronous workflows where the sender and receiver never talk directly.

The mechanics matter: when a consumer reads a message it becomes invisible for a visibility timeout rather than being deleted. The consumer must explicitly delete it after successful processing. If the consumer crashes, the timeout expires and the message reappears for another attempt, giving at-least-once delivery.

Queue storage is cheap, holds up to 500 TB, and messages can be up to 64 KB. Choose Service Bus instead when you need larger messages, ordering (FIFO sessions), topics with subscriptions, transactions, or duplicate detection; those are enterprise messaging features Queue storage lacks.

```
az storage message put --account-name mystore \  
-q jobs --content "process-order-42"
```

GOTCHA

Delivery is at-least-once, not exactly-once. A message can be processed twice if the visibility timeout expires before you delete it, so make your consumers idempotent.

46 Table storage

STORAGE

Q What is Azure Table storage and how is data keyed?

A Table storage is a NoSQL key-value store for structured, non-relational data. It is schemaless: each entity (row) is a bag of properties, and different entities in the same table can have different columns. The reason it scales cheaply is that it forgoes relational features like joins, foreign keys, and complex queries in favor of fast key-based lookups.

Every entity is identified by a composite key: a **PartitionKey** and a **RowKey**. The PartitionKey groups entities that are stored and served together, which is the unit of scalability and transaction scope; the RowKey uniquely identifies an entity within its partition. Queries that specify both keys are fast point lookups; queries that scan across partitions are slow.

For new projects the Cosmos DB Table API offers the same programming model with global distribution, guaranteed low latency, and automatic indexing, so it is often preferred over the classic Table storage service.

```
az storage entity insert --account-name mystore -t users \  
-e PartitionKey=US RowKey=42 name=Ana
```

GOTCHA

Batch (entity group) transactions only work within a single PartitionKey and are capped at 100 entities. Design partitions around what you need to update atomically, but avoid one giant hot partition.

47 Managed disks

STORAGE

Q What are managed disks and how do the disk types differ?

A Managed disks are block-storage volumes for Azure VMs where Azure handles the underlying storage account and placement for you. Before managed disks you had to manage storage accounts yourself and watch per-account IOPS limits; managed disks remove that burden and improve reliability by distributing disks across fault domains automatically.

There are four tiers by performance. **Standard HDD** is cheap, spinning-media, for backups and infrequent access. **Standard SSD** gives more consistent latency for light production. **Premium SSD** delivers high IOPS with a guaranteed SLA for most production workloads. **Ultra Disk** offers the highest, independently tunable IOPS and throughput for demanding databases like SAP HANA or top-tier SQL.

Premium SSD v2 and Ultra let you dial IOPS and throughput separately from capacity, so you don't have to overprovision size just to get performance. Disks are encrypted at rest by default.

```
az disk create -g rg -n data01 --size-gb 256 \  
  --sku Premium_LRS
```

GOTCHA

Not every VM size can attach every disk type. Premium and Ultra require compatible (often larger, 's'-suffixed) VM SKUs, and Ultra additionally requires enabling it in a specific zone. Check compatibility before you provision.

48 Blob lifecycle management

STORAGE

Q How does blob lifecycle management help control storage cost?

A Lifecycle management lets you define rules that automatically transition blobs between tiers or delete them based on age or last-access time. The value is that data cools over time, and paying hot-tier prices for logs no one has read in months is waste. Rules run automatically once a day, so you set policy once instead of writing cleanup scripts.

A rule is JSON with a filter (by prefix, blob type, or index tags) and actions keyed off conditions like **daysAfterModificationGreaterThan** or **daysAfterLastAccessTimeGreaterThan**. Typical policy: move to cool after 30 days, to archive after 90, and delete after 365.

This directly complements the access tiers: instead of manually calling set-tier on millions of blobs, the platform enforces your retention economics for you.

```
{ "enabled": true, "name": "cooldown",
  "definition": { "actions": { "baseBlob": {
    "tierToCool": { "daysAfterModificationGreaterThan": 30 },
    "delete": { "daysAfterModificationGreaterThan": 365 } } } } }
```

GOTCHA

Last-access-time tracking is not on by default and adds a small cost; enable it before writing rules that depend on it. Also, rules evaluate once per day, so transitions are not instantaneous.

Q What is a SAS token and how does it enable secure delegated access?

A A Shared Access Signature (SAS) is a signed URL that grants scoped, time-limited access to storage resources without exposing your account keys. The reason it matters: you often need to hand a client temporary read or write access to one blob, and giving them a full account key would be catastrophic. A SAS lets you delegate exactly the permission, resource, and window you intend.

There are three kinds. A **service SAS** grants access to one service, signed by the account key. An **account SAS** can span services. A **user delegation SAS** is signed by an Azure AD key instead of the account key, which is the most secure because it ties access to an identity and can be revoked by revoking the delegation.

Each SAS specifies permissions (read, write, list, delete), a start and expiry time, allowed IP ranges, and HTTPS-only. Stored access policies let you group SAS tokens so you can revoke them centrally.

```
az storage blob generate-sas --account-name mystore \  
-c data -n report.pdf --permissions r \  
--expiry 2026-08-07T00:00Z --https-only
```

GOTCHA

An ad-hoc SAS signed with the account key cannot be revoked before expiry short of rotating the key. Use short lifetimes, stored access policies, or a user delegation SAS so you retain a revocation path.

50 Azure SQL Database

DATABASES

Q What is Azure SQL Database and what does "PaaS" mean here?

A Azure SQL Database is a fully managed relational database service built on the SQL Server engine. As Platform-as-a-Service, Azure runs the OS, patching, backups, high availability, and the engine for you; you manage only the database, its schema, and its data. This removes most operational toil compared with running SQL Server on a VM.

It comes in deployment options: a **single database** with its own dedicated resources, or an **elastic pool** where multiple databases share a pool of resources to smooth out uneven, per-database load and save money. Built-in features include automatic backups with point-in-time restore, active geo-replication, auto-failover groups, and automatic tuning.

The trade-off for this convenience is reduced surface area: no SQL Agent, no cross-database queries in the classic sense, and no OS access. If an app needs those, SQL Managed Instance or SQL on a VM fits better.

```
az sql db create -g rg -s myserver -n appdb \  
  --service-objective S1
```

GOTCHA

The logical server (**myserver.database.windows.net**) is just a connection endpoint and security boundary, not a machine. It has no compute of its own; compute belongs to each database or elastic pool.

51 Managed Instance vs SQL DB vs SQL on VM

DATABASES

Q Compare SQL Managed Instance, Azure SQL Database, and SQL Server on a VM.

A These three sit on a spectrum from most managed to most controllable. The core decision is how much of SQL Server's full feature set and OS control you need versus how much operational work you want Azure to absorb. Moving toward the VM end gives you more control at the cost of more responsibility.

Azure SQL Database is the most managed: single databases or pools, best for new cloud-native apps, but with a reduced feature surface. **SQL Managed Instance** is also PaaS but offers near-100% compatibility with on-premises SQL Server, including SQL Agent, cross-database queries, CLR, and Service Broker, deployed inside your VNet. It is the sweet spot for lift-and-shift migrations that need instance-level features.

SQL Server on an Azure VM is IaaS: you get the full product and OS control, but you patch the OS, manage backups, and configure HA yourself. Choose it when you need OS-level access, a specific SQL version, or features neither PaaS option supports.

```
az sql mi create -g rg -n myinstance \  
  --subnet /subscriptions/.../subnets/miSubnet \  
  --capacity 8 --edition GeneralPurpose
```

GOTCHA

Managed Instance deploys into a delegated subnet and takes a while (often hours) to provision or scale because it stands up a dedicated virtual cluster. Plan the network and timing before a migration cutover.

52 DTU vs vCore

DATABASES

Q What is the difference between the DTU and vCore purchasing models?

A These are two ways to buy and size Azure SQL Database compute. The DTU model bundles CPU, memory, and I/O into a single blended metric, while the vCore model unbundles them so you choose the number of virtual cores and memory independently. The reason vCore exists is transparency and flexibility: you can align capacity with real hardware and control costs precisely.

DTU (Database Transaction Unit) offers simple tiers (Basic, Standard, Premium) where a higher number means proportionally more of everything. It is easy but opaque; you cannot tune CPU separately from I/O. **vCore** exposes hardware generations and lets you pick General Purpose, Business Critical, or Hyperscale service tiers.

vCore also unlocks two big cost levers: Azure Hybrid Benefit lets you reuse existing SQL Server licenses, and reserved capacity gives discounts for commitment. For this reason Microsoft recommends vCore for most new and larger workloads.

```
az sql db create -g rg -s myserver -n appdb \  
  --edition GeneralPurpose --family Gen5 --capacity 4
```

GOTCHA

Only the vCore model supports Hyperscale (up to 100 TB with fast restores) and serverless auto-pause billing. If you might need those, don't box yourself in with DTU.

53 Azure Cosmos DB

DATABASES

Q What is Azure Cosmos DB and what problem does it solve?

A Cosmos DB is a globally distributed, multi-model NoSQL (and now relational-style) database designed for low latency at any scale. Its reason for being is turnkey global distribution: you add regions with a click and Cosmos replicates data everywhere, giving single-digit-millisecond reads and writes close to users, backed by comprehensive SLAs on latency, throughput, availability, and consistency.

It is multi-model through multiple APIs over the same engine: the native **NoSQL (Core)** API, plus **MongoDB**, **Cassandra**, **Gremlin** (graph), and **Table** APIs, so you can lift existing apps that speak those protocols.

Throughput is provisioned in Request Units per second (RU/s), and everything is automatically indexed. You can enable multi-region writes for active-active topologies where every region accepts writes, which is what makes it a good fit for planet-scale, always-on applications.

```
az cosmosdb create -g rg -n mycosmos \  
  --locations regionName=eastus --locations regionName=westeurope \  
  --enable-multiple-write-locations true
```

GOTCHA

Cosmos bills for provisioned RU/s whether or not you use them (unless you pick serverless or autoscale). An idle over-provisioned container quietly burns money; use autoscale or serverless for spiky and dev workloads.

54 Cosmos DB consistency levels

DATABASES

Q Explain the five Cosmos DB consistency levels.

A Cosmos DB offers five consistency levels that let you trade off between read correctness and latency/availability. This is unusual: most databases give you one. The reason is the CAP/PACELC reality of global replication, where waiting for all replicas to agree adds latency, so Cosmos lets you choose exactly how much staleness you tolerate.

From strongest to weakest: **Strong** guarantees linearizable, always-latest reads but only within a single region write scope and at the highest latency. **Bounded staleness** lets reads lag by at most a configured number of versions or a time window. **Session** (the default) guarantees consistency within a single client session, so a user always reads their own writes. **Consistent prefix** guarantees you never see writes out of order. **Eventual** gives the lowest latency with no ordering guarantee.

Weaker levels also cost fewer RUs for reads, so the choice affects both correctness and price. Most apps find **Session** the right balance.

```
az cosmosdb update -g rg -n mycosmos \  
  --default-consistency-level Session
```

GOTCHA

Strong consistency is incompatible with multi-region writes: you cannot have every region accept writes and still guarantee globally linearizable reads. Choosing Strong constrains your topology.

55 Cosmos DB partitioning and RU/s

DATABASES

Q How do partitioning and Request Units work in Cosmos DB?

A Cosmos DB scales horizontally by spreading data across physical partitions, and it meters all work in Request Units per second (RU/s). Understanding both is essential because a poor partition key or under-provisioned RU/s is the number-one cause of throttling and cost surprises. The two are linked: RU/s are divided evenly across partitions.

You choose a **partition key** (a property like `/userId`) that determines which logical partition an item belongs to. Cosmos maps logical partitions onto physical ones. A good key has high cardinality and spreads reads and writes evenly; a bad key creates a hot partition that bottlenecks throughput regardless of total provisioned RU/s.

Every operation costs RUs: a point read is about 1 RU, queries and writes cost more. If your workload exceeds provisioned RU/s you get throttled (HTTP 429) and must retry. Autoscale RU/s scales between 10% and 100% of a max automatically.

```
az cosmosdb sql container create -g rg -a mycosmos \  
  -d appdb -n orders --partition-key-path /userId \  
  --max-throughput 4000
```

GOTCHA

The partition key is immutable after creation. Since total RU/s is split evenly across physical partitions, one hot logical partition can throttle even when the account looks under-utilized overall. Choose the key carefully up front.

56 Azure Database for PostgreSQL / MySQL

DATABASES

Q What are Azure Database for PostgreSQL and MySQL, and why use them?

A These are managed PaaS offerings of the open-source PostgreSQL and MySQL engines. The appeal is that you get the exact community database your app already uses, with no code changes, while Azure handles patching, automated backups, high availability, and scaling. You keep engine compatibility but shed the operational burden.

The current recommended deployment for PostgreSQL is **Flexible Server**, which gives you zone-redundant HA, control over maintenance windows, and stop/start to save cost on non-production. MySQL likewise offers Flexible Server. (The older Single Server option is being retired.)

For PostgreSQL, Azure Cosmos DB for PostgreSQL (formerly Hyperscale/Citus) adds sharding across nodes for horizontal scale-out when a single server is not enough. Choose these services when you want open-source compatibility rather than the SQL Server engine.

```
az postgres flexible-server create -g rg -n mypg \  
  --tier Burstable --sku-name Standard_B1ms --version 16
```

GOTCHA

Managed PostgreSQL/MySQL do not give you superuser or OS access, so extensions must be on Azure's allow-list and some server parameters are locked. Verify your required extensions are supported before migrating.

57 Azure Cache for Redis

DATABASES

Q What is Azure Cache for Redis and what are its common patterns?

A Azure Cache for Redis is a managed, in-memory data store based on Redis, used to accelerate applications by keeping hot data in RAM. The reason it dramatically improves performance is that reading from memory is orders of magnitude faster than hitting a disk-backed database, and it offloads repetitive read traffic from your primary store.

Common patterns include **cache-aside** (the app checks Redis first and falls back to the database on a miss, then populates the cache), **session store** for stateless web farms, **distributed locking**, and lightweight **pub/sub** messaging. Because it also supports data structures like sorted sets, it is popular for leaderboards and rate limiting.

Tiers range from Basic (single node, no SLA) through Standard (replicated), Premium (clustering, persistence, VNet), up to Enterprise tiers with Redis modules and active geo-replication.

```
az redis create -g rg -n mycache \  
  --sku Standard --vm-size c1 --location eastus
```

GOTCHA

A cache is not durable by default; treat it as disposable. If you store the only copy of important data in Basic or Standard tier, a node restart can lose it. Use Premium persistence or keep the source of truth elsewhere.

58 Data Lake Storage Gen2

STORAGE

Q What is Azure Data Lake Storage Gen2 and how does it differ from plain Blob?

A Data Lake Storage Gen2 (ADLS Gen2) is not a separate service but a set of capabilities layered onto a Blob storage account, aimed at big-data analytics. The defining addition is the **hierarchical namespace**, which turns the flat blob container into real directories. This makes operations like renaming or deleting a folder atomic and cheap instead of touching every object.

The hierarchical namespace also enables POSIX-style access control lists (ACLs) on files and folders, on top of Azure RBAC, giving fine-grained permissions that analytics frameworks expect. ADLS Gen2 exposes a dedicated **dfs** endpoint and is optimized for high-throughput parallel reads.

It integrates natively with analytics engines like Azure Synapse, Databricks, and HDInsight. Choose it as the storage foundation for data lakes and analytics; use ordinary Blob for general object storage and web content.

```
az storage account create -n mylake -g rg \  
  --sku Standard_LRS --kind StorageV2 \  
  --enable-hierarchical-namespace true
```

GOTCHA

The hierarchical namespace must be enabled at account creation and cannot be toggled on later. If you provision a plain blob account and then need ADLS features, you must create a new account and migrate.

59 Backup and point-in-time restore

DATABASES

Q How does point-in-time restore work in Azure SQL, and how does it relate to backups?

A Azure SQL Database automatically takes backups: full, differential, and continuous transaction-log backups. The reason this enables point-in-time restore (PITR) is that by replaying transaction logs on top of a full backup, the service can reconstruct the database as it existed at any second within the retention window, not just at backup snapshots.

The default retention is 7 days (configurable up to 35 for short-term, and years with long-term retention policies). A restore always creates a **new** database rather than overwriting the existing one, which protects you from making a bad situation worse. This is your primary recovery from accidental deletes or bad deployments.

PITR is distinct from geo-redundancy: GRS protects against regional loss but faithfully replicates mistakes, whereas PITR lets you rewind time. Use both together. For broader coverage, Azure Backup and Recovery Services vaults centralize backup for VMs, files, and databases.

```
az sql db restore -g rg -s myserver -n appdb \  
  --dest-name appdb-restored --time 2026-08-06T09:30:00
```

GOTCHA

Restore creates a new database and can take significant time proportional to database size; it is not instant. Because you cannot restore in place, plan a rename/swap step in your recovery runbook.

60 Azure SQL vs Cosmos DB

DATABASES

Q How do you choose between Azure SQL Database and Cosmos DB?

A The decision hinges on your data model and consistency needs versus your scale and global-distribution needs. Choose Azure SQL when your data is relational, benefits from joins, foreign keys, and ACID transactions across tables, and your workload fits comfortably within a region or a few. Choose Cosmos DB when you need elastic horizontal scale, single-digit-millisecond latency, and turnkey multi-region, active-active writes over a flexible schema.

Azure SQL shines for line-of-business apps, reporting, and anything with a rich relational schema and strong transactional integrity. Cosmos DB shines for high-velocity, high-volume, globally distributed workloads like IoT telemetry, catalogs, personalization, and gaming, where a fixed schema and joins would get in the way.

Cost models differ too: SQL bills by DTU or vCore compute, while Cosmos bills by provisioned or consumed RU/s plus storage. They are not mutually exclusive; many architectures use Azure SQL as the transactional system of record and Cosmos for the globally distributed serving layer.

```
# rule of thumb
relational + joins + strong ACID  -> Azure SQL
global scale + low latency + flexible schema -> Cosmos DB
```

GOTCHA

Don't pick Cosmos just because it says "NoSQL scale." If your access patterns need ad-hoc joins and multi-row transactions, forcing them onto Cosmos means reimplementing relational logic in the app, which is costly and error-prone.

61 Virtual Network and subnets

NETWORKING

Q What is an Azure Virtual Network (VNet) and how do subnets fit in?

A A VNet is your private, isolated slice of the Azure network fabric, defined by one or more private address spaces in CIDR notation (for example 10.0.0.0/16). It is the fundamental building block that lets your resources communicate securely with each other, the internet, and on-premises networks. The WHY: Azure gives every VNet its own isolated layer-3 boundary, so traffic never leaks between tenants, and you control routing, filtering, and connectivity entirely.

A subnet carves the VNet address space into smaller ranges (for example 10.0.1.0/24), and resources like VMs, App Service integration, or private endpoints are deployed into a specific subnet. Subnets let you apply Network Security Groups, route tables, and service delegation at a finer grain. A VNet is regional and can span all availability zones in that region, but it cannot span regions.

Azure reserves five addresses per subnet: the network and broadcast addresses plus three for the default gateway and DNS mapping, so a /24 gives you 251 usable IPs, not 256. Plan address space to avoid overlap, especially if you will peer VNets or connect to on-premises.

```
az network vnet create -g rg -n vnet1 --address-prefix 10.0.0.0/16 \  
  --subnet-name web --subnet-prefix 10.0.1.0/24
```

GOTCHA

Overlapping address spaces break VNet peering and VPN connections. Never reuse the same CIDR across VNets you intend to connect.

62 Network Security Groups

NETWORKING

Q How does a Network Security Group filter traffic?

A A Network Security Group (NSG) is a stateful packet filter holding a prioritized list of allow/deny rules matched on source, destination, port, and protocol. The WHY behind "stateful": if you allow an inbound flow, the return traffic is automatically permitted without a matching outbound rule, which dramatically simplifies rule sets compared to classic ACLs.

Rules are evaluated by priority (lower number wins), from 100 to 4096. Azure adds default rules that allow VNet-internal traffic and outbound internet, and deny everything else; you cannot delete defaults but you can override them with higher-priority custom rules. You can attach an NSG to a subnet, a NIC, or both, and both sets are evaluated.

Use Service Tags like `AzureLoadBalancer` or `Internet` and Application Security Groups to reference groups of resources by role instead of hardcoding IPs, which keeps rules readable as infrastructure scales.

```
az network nsg rule create -g rg --nsg-name nsg1 -n AllowHTTPS \  
  --priority 100 --direction Inbound --access Allow \  
  --protocol Tcp --destination-port-ranges 443 --source-address-prefixes  
Internet
```

GOTCHA

When an NSG is on both subnet and NIC, inbound traffic must pass the subnet NSG first then the NIC NSG; a deny at either layer drops the packet.

Q What does the Azure Load Balancer do and at what layer?

A Azure Load Balancer operates at layer 4 (TCP/UDP), distributing inbound flows across a backend pool of VMs using a hash of the 5-tuple (source IP, source port, destination IP, destination port, protocol). The WHY: because it works at the transport layer, it is extremely fast and protocol-agnostic, but it cannot inspect URLs, headers, or cookies.

It comes in Public (internet-facing) and Internal (private VIP) flavors. Health probes continuously check backend instances and remove unhealthy ones from rotation. The Standard SKU is zone-redundant, secure by default (traffic denied unless an NSG allows it), and required for availability zones.

Load balancing rules map a frontend port to a backend port; you can also configure inbound NAT rules to forward a specific port to a single VM (useful for SSH/RDP jump access). It scales to millions of flows with no per-flow latency penalty.

```
az network lb create -g rg -n lb1 --sku Standard \
  --frontend-ip-name fe --backend-pool-name bepool
```

GOTCHA

Basic and Standard SKUs cannot mix in one deployment, and Basic is being retired. Choose Standard for any production or zone-redundant workload.

Q When would you use Application Gateway with WAF?

A Application Gateway is a layer 7 (HTTP/HTTPS) regional load balancer that understands the application protocol, so it can route on URL path and host header, terminate TLS, offload SSL, inject cookies for session affinity, and rewrite headers. The WHY: layer 7 awareness lets you send `/images/` to one backend pool and `/api/` to another from a single public IP.

The optional Web Application Firewall (WAF) tier adds protection against common exploits using the OWASP Core Rule Set, blocking SQL injection, cross-site scripting, and more before traffic reaches your app. WAF can run in Detection mode (log only) or Prevention mode (block).

It supports autoscaling (v2 SKU), zone redundancy, and end-to-end TLS. Because it is regional, pair it with Front Door or Traffic Manager for global distribution.

```
az network application-gateway create -g rg -n appgw \  
  --sku WAF_v2 --capacity 2 --waf-policy wafpol \  
  --http-settings-protocol Https --frontend-port 443
```

GOTCHA

App Gateway is regional, not global. For multi-region failover put Front Door or Traffic Manager in front of per-region gateways.

65 Azure Front Door

NETWORKING

Q What problem does Azure Front Door solve?

A Azure Front Door is a global, layer 7 entry point that combines a content delivery network, anycast routing, TLS termination at the edge, and a WAF. The WHY: it uses Microsoft's global edge network so users connect to the nearest point of presence, and Front Door routes their request over Microsoft's backbone to the healthiest, closest backend, cutting latency and improving resilience.

It provides global HTTP load balancing with instant failover, split TCP to accelerate connection setup, caching of static content at the edge, and URL-based routing. Unlike Traffic Manager it terminates the connection at the edge, so failover is near-instant rather than waiting on DNS TTLs.

The Standard and Premium tiers unify CDN and WAF; Premium adds private origin connectivity via Private Link and managed bot/security rules. Backends can be App Service, Storage static sites, or any public origin.

```
az afd profile create -g rg --profile-name fd1 --sku Premium_AzureFrontDoor
```

GOTCHA

Front Door only handles HTTP/HTTPS. For non-HTTP protocols (raw TCP/UDP) use Traffic Manager or a global Load Balancer instead.

66 Traffic Manager (DNS-based)

NETWORKING

Q How does Azure Traffic Manager route users?

A Traffic Manager is a DNS-based global traffic router. It does not sit in the data path; instead it responds to DNS queries with the IP of the best endpoint according to a routing method, and the client then connects directly to that endpoint. The WHY: because it only touches DNS, it works with any protocol (not just HTTP) and adds zero data-plane latency, but failover is bounded by DNS TTL and client caching.

Routing methods include Priority (active/passive failover), Weighted (percentage split), Performance (lowest latency), Geographic (route by user location for compliance), MultiValue, and Subnet. Endpoint health probing removes unhealthy endpoints from DNS responses.

It is ideal for routing across regions or even across cloud/on-premises endpoints. Combine it with regional Load Balancers or App Gateways as the actual endpoints.

```
az network traffic-manager profile create -g rg -n tml \  
  --routing-method Performance --unique-dns-name myapp
```

GOTCHA

Because clients cache DNS, failover is not instant. Lower the TTL to reduce failover time, but you cannot beat DNS caching entirely; use Front Door when sub-second failover matters.

Q How do you choose between Load Balancer, App Gateway, Front Door, and Traffic Manager?

A A: Decide along two axes: global vs regional, and HTTP(S) vs any protocol. The WHY: Azure deliberately splits these because a single service cannot optimize both global anycast routing and deep layer-7 inspection at once, so you compose them.

Regional + non-HTTP use Load Balancer (L4, TCP/UDP). **Regional + HTTP** use Application Gateway (L7, WAF, path routing). **Global + HTTP** use Front Door (edge, CDN, WAF, fast failover). **Global + non-HTTP** use Traffic Manager (DNS, any protocol).

They compose naturally: Front Door globally in front of per-region Application Gateways, each fronting an internal Load Balancer for a VM scale set. Pick the simplest set that meets your latency, protocol, and failover requirements.

GOTCHA

A common wrong answer is "use Front Door for everything." Front Door cannot handle non-HTTP traffic and adds cost/complexity you may not need for a single-region internal app.

68 VNet peering

NETWORKING

Q What is VNet peering and how does traffic flow?

A VNet peering connects two VNets so resources communicate using private IPs as if on the same network, with traffic staying on the Microsoft backbone (never the public internet). The WHY: peering is implemented in the Azure fabric itself, so it delivers low-latency, high-bandwidth connectivity with no gateway or VPN in the path.

Peering can be regional (same region) or global (across regions). It is non-transitive: if A peers with B and B peers with C, A cannot reach C automatically; you either peer A-C directly or route through a hub using a network virtual appliance and gateway transit. Address spaces must not overlap.

Options like "allow gateway transit" and "use remote gateways" let spoke VNets share a single VPN/ExpressRoute gateway in a hub, the basis of the hub-and-spoke topology.

```
az network vnet peering create -g rg -n hubToSpoke \  
  --vnet-name hub --remote-vnet spoke --allow-vnet-access
```

GOTCHA

Peering is not transitive. Spoke-to-spoke traffic through a hub requires user-defined routes pointing at a firewall or NVA in the hub.

Q What is an Azure VPN Gateway used for?

A A VPN Gateway sends encrypted traffic between an Azure VNet and an on-premises network (site-to-site) or individual clients (point-to-site) over the public internet using IPsec/IKE. The WHY: it gives you private connectivity without dedicated circuits, trading some throughput and latency predictability for low cost and quick setup.

Site-to-site connects your on-prem VPN device to Azure; point-to-site lets remote developers tunnel in from laptops; VNet-to-VNet links two VNets across regions. Throughput and tunnel count depend on the gateway SKU (VpnGw1 through VpnGw5), and active-active configurations improve resilience.

It lives in a dedicated **GatewaySubnet** and takes tens of minutes to provision. For higher, more consistent bandwidth or private (non-internet) connectivity, ExpressRoute is the alternative.

```
az network vnet-gateway create -g rg -n vpngw \  
  --vnet vnet1 --gateway-type Vpn --sku VpnGw2 --vpn-type RouteBased
```

GOTCHA

The subnet must be named exactly **GatewaySubnet** and should be at least /27. A wrong name or too-small range makes gateway creation fail.

70 ExpressRoute

NETWORKING

Q How is ExpressRoute different from a VPN Gateway?

A ExpressRoute establishes a private, dedicated connection between your on-premises network and Azure through a connectivity provider, bypassing the public internet entirely. The WHY: because traffic never touches the internet, you get predictable latency, higher and guaranteed bandwidth, and better security than an IPsec VPN over the public net.

It uses BGP to exchange routes and offers peering types: private peering (to VNets) and Microsoft peering (to Microsoft 365 and PaaS public endpoints). Bandwidth ranges from 50 Mbps to 100 Gbps, and it can be made zone-redundant and globally reachable via ExpressRoute Global Reach.

It is more expensive and slower to provision (involves a carrier) than a VPN, so it suits enterprises with heavy, latency-sensitive, or compliance-driven hybrid workloads. Many designs use a VPN as an encrypted backup path for the ExpressRoute circuit.

GOTCHA

ExpressRoute is private but not encrypted by default. For regulated data, layer IPsec (or MACsec) on top; do not assume the circuit itself provides confidentiality.

71 Private Endpoints and Private Link

NETWORKING

Q What do Private Endpoints and Private Link achieve?

A A Private Endpoint is a network interface with a private IP from your VNet that maps to a specific Azure PaaS resource (a storage account, SQL database, Key Vault, etc.), delivered by the Azure Private Link service. The WHY: it pulls a normally public PaaS service onto your private network so traffic never traverses the internet and the service's public endpoint can be disabled entirely.

DNS is the critical piece: the resource's public FQDN must resolve to the private IP, typically via an Azure Private DNS zone linked to your VNet. Without correct DNS, clients still resolve the public IP and bypass the private endpoint.

Private Link also lets you expose your own service privately to consumers in other tenants. It reduces data exfiltration risk and simplifies compliance because access is scoped to specific resources, not entire service ranges.

```
az network private-endpoint create -g rg -n pe-sql --vnet-name vnet1 \
  --subnet data --private-connection-resource-id
"/subscriptions/.../servers/sql1" \
  --group-id sqlServer --connection-name c1
```

GOTCHA

Forgetting the Private DNS zone is the classic failure: name resolution still returns the public IP, so connections silently ignore the private endpoint.

72 Azure Firewall

NETWORKING

Q What is Azure Firewall and how does it differ from an NSG?

A Azure Firewall is a managed, stateful, cloud-native network firewall with built-in high availability and unrestricted scaling. Unlike an NSG (a basic layer 3/4 allow-deny filter attached to subnets or NICs), the firewall centralizes policy, supports FQDN-based application rules, threat intelligence feeds, and full traffic logging. The WHY: NSGs micro-segment inside a VNet, while Azure Firewall governs north-south traffic at a hub for an entire hub-and-spoke network.

It provides network rules (IP/port/protocol), application rules (allow outbound to specific FQDNs like `*.microsoft.com`), NAT rules for inbound DNAT, and threat intelligence-based filtering. It sits in a dedicated `AzureFirewallSubnet` in the hub, and spoke routes point default traffic to it.

The Premium SKU adds TLS inspection, an intrusion detection and prevention system (IDPS), and URL filtering. Use Firewall and NSGs together, not as either/or.

GOTCHA

Azure Firewall is not a WAF. It protects network egress/ingress but does not inspect HTTP payloads for OWASP attacks; use Application Gateway WAF or Front Door WAF for that.

73 DDoS Protection

NETWORKING

Q How does Azure DDoS Protection work?

A Azure DDoS Protection defends public-facing resources against volumetric, protocol, and resource-layer attacks. Every Azure workload gets always-on Basic (Infrastructure) protection for free, absorbing large-scale attacks at the platform edge. The WHY: attackers try to exhaust bandwidth or connection state, so mitigation must happen at Microsoft's network edge before traffic reaches your resources.

The paid Network Protection (formerly Standard) tier adds adaptive tuning based on your traffic baseline, per-resource mitigation policies, attack metrics and alerts, and cost protection credits for scale-out during an attack. It applies to public IPs in protected VNets.

Pair it with a WAF for layer 7 attacks: DDoS Protection handles volumetric floods, while the WAF blocks application-layer exploits like HTTP floods targeting expensive endpoints.

GOTCHA

Basic tier gives no per-resource tuning, telemetry, or alerts. If you need attack visibility and SLA-backed mitigation, you must enable the paid Network Protection plan.

74 Microsoft Entra ID

SECURITY

Q What is Microsoft Entra ID and how does it relate to a subscription?

A A: Microsoft Entra ID (formerly Azure Active Directory) is Azure's cloud identity and access management service. It authenticates users, groups, and applications and issues tokens; Azure resources then use those identities for authorization. The WHY: Entra ID is the trust anchor, it establishes who you are, and Azure RBAC separately decides what you can do.

An Entra tenant is a dedicated identity boundary. A subscription trusts exactly one tenant for authentication, but a tenant can back many subscriptions. Entra ID is not the same as on-premises Active Directory Domain Services; it is HTTP/REST-based (OAuth 2.0, OpenID Connect, SAML) rather than Kerberos/LDAP, though you can sync between them with Entra Connect.

It underpins single sign-on, Conditional Access, MFA, managed identities, and B2B/B2C collaboration. Licensing tiers (Free, P1, P2) unlock features like Conditional Access and Identity Protection.

GOTCHA

Entra ID handles authentication (identity), not authorization to resources; granting someone a role still requires an Azure RBAC assignment. Conflating the two is a common interview slip.

75 Azure RBAC

SECURITY

Q How does Azure RBAC control access, and what are scopes?

A Azure Role-Based Access Control grants permissions by assigning a role to a security principal (user, group, service principal, or managed identity) at a scope. The three pieces are the who (principal), the what (role definition, a set of allowed actions), and the where (scope). The WHY: this triple lets you grant least-privilege access precisely where needed.

Scope is hierarchical: management group to subscription to resource group to resource. Assignments inherit downward, so a Reader at the subscription is a Reader on every resource group and resource inside it. Built-in roles like Owner, Contributor, and Reader cover most needs; custom roles let you tailor exact actions.

RBAC is additive and allow-based, but explicit deny assignments can override an allow. Effective permissions are the union of all role assignments across inherited scopes.

```
az role assignment create --assignee user@contoso.com \  
  --role Reader --scope /subscriptions/00000000-.../resourceGroups/rg
```

GOTCHA

Contributor can manage resources but cannot grant access to others; only Owner and User Access Administrator can create role assignments. Watch for that distinction.

76 Managed identities

SECURITY

Q What are managed identities and how do system vs user assigned differ?

A A managed identity is an Entra ID identity that Azure automatically manages for a resource, so your code can authenticate to services like Key Vault or Storage without storing any credentials. The WHY: it eliminates secrets in code and config, Azure rotates the underlying credentials for you, removing the biggest cause of leaked keys.

A system-assigned identity is created on and tied to a single resource; its lifecycle matches that resource and it is deleted when the resource is deleted. A user-assigned identity is a standalone resource you create once and attach to many resources, and it survives independently, which is ideal when several apps need the same identity or you want to pre-grant permissions before deployment.

Either way, the app requests a token from the local identity endpoint and uses it against Azure services, with RBAC controlling what the identity may do.

```
az vm identity assign -g rg -n vm1 # system-assigned
az identity create -g rg -n myUAI # reusable user-assigned
```

GOTCHA

System-assigned identities vanish with the resource, breaking any role assignments referencing them. For shared or long-lived permissions, prefer user-assigned.

77 Service principals

SECURITY

Q What is a service principal and when do you use one?

A A service principal is the local identity that an application or automation uses to authenticate and be authorized in an Entra tenant. It is the concrete instance (in one tenant) of an app registration's global object. The WHY: humans use user accounts, but scripts, CI/CD pipelines, and apps need their own non-human identity to sign in and get RBAC-scoped access.

You authenticate a service principal with a client secret or, more securely, a certificate, then assign it roles just like a user. This is how tools like Terraform or GitHub Actions deploy to Azure from outside the platform.

A managed identity is essentially a special service principal whose credentials Azure manages for you. So prefer managed identities for workloads running inside Azure, and reserve manually-created service principals for external systems that cannot use one.

```
az ad sp create-for-rbac -n ci-deployer \  
  --role Contributor --scopes /subscriptions/.../resourceGroups/rg
```

GOTCHA

Client secrets expire and leak. Whenever the workload runs inside Azure, use a managed identity instead of a service principal secret.

78 Azure Key Vault

SECURITY

Q What is Azure Key Vault and how is access controlled?

A Azure Key Vault is a managed service for securely storing and controlling access to secrets, keys, and certificates. Secrets hold passwords and connection strings, keys are cryptographic keys for encryption/signing (optionally HSM-backed), and certificates manage TLS certs with auto-renewal. The WHY: centralizing sensitive material means apps fetch it at runtime via a managed identity, so nothing sensitive lives in code or config.

Access is governed by either Azure RBAC (recommended) or the older vault access policies. Every operation can be logged for audit, and soft-delete plus purge protection guard against accidental or malicious deletion by retaining deleted items for a recovery window.

The Premium SKU provides FIPS 140-2 Level 2 HSM-backed keys. Combine with private endpoints to keep vault traffic off the public internet.

```
az keyvault secret set --vault-name kv1 -n dbpass --value "s3cr3t"
```

GOTCHA

Enable purge protection in production. Without it, a soft-deleted vault or key can be permanently purged, and there is no undo.

79 Azure Bastion

SECURITY

Q What is Azure Bastion and why use it?

A Azure Bastion is a managed PaaS service that provides secure RDP and SSH connectivity to your VMs directly through the Azure portal over TLS, without exposing the VMs' public IPs or opening RDP/SSH ports to the internet. The WHY: publicly reachable 3389/22 ports are a top attack vector, and Bastion removes them entirely by brokering the session from inside the VNet.

You deploy Bastion into a dedicated `AzureBastionSubnet`, and it connects to VMs using their private IPs. Users reach it over HTTPS (port 443), so no VPN client or public VM IP is required. It also protects against port scanning since VMs stay private.

Higher SKUs add native client support, IP-based connections, and shareable links. It complements just-in-time VM access and NSG hardening.

GOTCHA

The subnet must be named exactly `AzureBastionSubnet` and be at least /26. A smaller or misnamed subnet blocks deployment.

Q How do Conditional Access and MFA work together?

A Conditional Access is Entra ID's policy engine that evaluates signals at sign-in and enforces controls, effectively if-then rules: if a user in a certain group signs in from an untrusted location to a sensitive app, then require multi-factor authentication or block. The WHY: static passwords are weak, so access decisions should adapt to real-time risk signals like device state, location, and sign-in risk.

MFA requires a second factor (authenticator app, FIDO2 key, phone) beyond the password, sharply reducing account-takeover risk. Conditional Access is the mechanism that decides when MFA (or device compliance, or a block) is demanded, rather than forcing it on every sign-in.

Signals include user/group, application, location, device compliance, and Identity Protection risk level (P2). Grant controls range from require MFA to require a compliant device or block access. Always configure emergency break-glass accounts excluded from policies.

GOTCHA

A Conditional Access policy that requires MFA for all users with no break-glass exclusion can lock every admin out. Always keep excluded emergency accounts.

81 Azure Monitor: metrics vs logs

MONITORING

Q What is Azure Monitor, and how do metrics differ from logs?

Azure Monitor is the umbrella platform that collects telemetry from every layer of your estate: platform resources, guest OS, applications, and the Azure infrastructure itself. Understanding it means understanding its two fundamentally different data stores, because they have different shapes, costs, and query engines.

Metrics are numeric, time-series data collected at regular intervals and stored in a purpose-built time-series database. They are lightweight, near-real-time (typically 1-minute granularity), and cheap. Examples: **Percentage CPU**, **Disk Read Bytes**, request latency. You query them with the metrics explorer and alert on them with sub-minute latency.

Logs are richer, structured or unstructured event records stored in a Log Analytics workspace and queried with KQL. They can hold text, dimensions, and complex relationships, but ingestion has cost and a few-minute latency. Use logs when you need context ("which user hit this error?"), metrics when you need cheap, fast numeric trending.

A common design is to use platform metrics for autoscale and fast alerts, and route diagnostic logs to a workspace for investigation. Many resources emit both, and you can even convert log data into metrics (metric alerts on log queries) when you need the richness of logs with the speed of metrics.

GOTCHA

Platform metrics are retained only 93 days by default. If you need long-term numeric history, route them to a Log Analytics workspace or a storage account via a diagnostic setting.

82 Log Analytics workspaces

MONITORING

Q What is a Log Analytics workspace, and how should you design your workspace topology?

A Log Analytics workspace is the container that stores log and performance data and provides the KQL query engine over it. It is the destination for diagnostic settings, Application Insights (workspace-based), Microsoft Sentinel, and VM agents. Every table (like `Heartbeat` or `AzureActivity`) lives inside a workspace, and the workspace is the boundary for retention, access control, and billing.

The central design question is **how many workspaces** to run. The default guidance is to centralize into as few workspaces as possible, because cross-workspace queries add friction and some features (like Sentinel correlation) work best with co-located data. Reasons to split include data-sovereignty (region-specific workspaces), separate RBAC or retention needs, and cost chargeback per business unit.

Access can be scoped two ways: **workspace-context** (permissions on the workspace grant access to all its data) and **resource-context** (users query logs through the resource they own, seeing only that resource's rows). Resource-context RBAC is what lets a team see their VM's logs without granting access to the whole workspace.

Heartbeat

```
| where TimeGenerated > ago(30m)
| summarize LastSeen = max(TimeGenerated) by Computer
| where LastSeen < ago(10m) // machines that went silent
```

GOTCHA

Retention is set per-workspace but can be overridden per-table. Long retention on high-volume tables is a top hidden cost; use Basic/Auxiliary logs tiers and Archive for cheap, rarely-queried data.

83 Application Insights

MONITORING

Q What is Application Insights and what does it give you that platform metrics do not?

Application Insights is the application performance management (APM) feature of Azure Monitor. Where platform metrics tell you the CPU of a VM, App Insights tells you what your **code** is doing: request rates and durations, dependency calls, exceptions, traces, and custom events, all correlated by an operation ID so you can follow a single user request across services.

You instrument an app either with the SDK, auto-instrumentation (codeless attach for App Service, Functions, AKS), or OpenTelemetry. The killer feature is **distributed tracing**: because each request carries a correlation ID, the Application Map and end-to-end transaction view stitch together the browser call, the API, the database dependency, and the downstream service, so you can see exactly where latency or failures originate.

Key tables in the workspace are **requests**, **dependencies**, **exceptions**, **traces**, and **customEvents**. Live Metrics gives you a real-time firehose during a deployment, and Smart Detection flags anomalies like a sudden rise in failure rate without you writing an alert rule.

```
requests
| where timestamp > ago(1h)
| summarize p95 = percentile(duration, 95), failures = countif(success == "False") by name
| order by p95 desc
```

GOTCHA

Prefer workspace-based Application Insights; classic (standalone) resources are retired. Watch sampling: adaptive sampling drops telemetry under load, so raw counts in KQL can understate true volume unless you use **itemCount** to reweight.

84 KQL basics

MONITORING

Q What are the fundamentals of Kusto Query Language (KQL)?

A QL is the read-only query language for Log Analytics, Application Insights, Sentinel, and Azure Data Explorer. Its mental model is a **pipeline**: you start with a table and pipe (|) it through a sequence of operators, each transforming the tabular result of the previous one. This left-to-right flow is why KQL reads naturally top to bottom.

The core operators you use daily are where (filter rows), project (select/compute columns), extend (add columns), summarize (aggregate, with by for grouping), order by (sort), top (limit), and join. Time filtering with `TimeGenerated > ago(1h)` should come early so the engine prunes data before doing expensive work.

Performance is a real skill: filter early, avoid `contains` when `has` (token-indexed) will do, and prefer `summarize` over pulling raw rows. The `bin()` function buckets timestamps for time-series charts, and `render` produces a visualization inline.

```
AzureActivity
| where TimeGenerated > ago(24h)
| where ActivityStatusValue == "Failure"
| summarize count() by bin(TimeGenerated, 1h), OperationNameValue
| render timechart
```

GOTCHA

KQL string comparisons are case-sensitive with `==` but case-insensitive with `==~`. A frequent bug is filtering `ResultType == "success"` when the data stores "Success"; use `==~` or normalize with `tolower()`.

85 Alerts and action groups

MONITORING

Q How do Azure alerts work, and what role do action groups play?

An Azure Monitor **alert rule** couples three things: a target resource, a condition (a signal crossing a threshold), and one or more **action groups** that fire when the alert changes state. Separating the "what to detect" from the "who to notify" is the key design idea, because it lets many rules reuse the same notification and remediation logic.

There are several signal types. **Metric alerts** evaluate numeric metrics with low latency and support dynamic thresholds that learn normal behavior. **Log alerts** run a KQL query on a schedule and fire when results cross a threshold. **Activity log alerts** watch control-plane events like a resource being deleted or a service-health incident.

An **action group** is a reusable bundle of actions: email/SMS/push, webhooks, Logic Apps, Azure Functions, ITSM connectors, and automation runbooks. Because it is reusable, you define your on-call routing once and attach it to dozens of rules. Alert processing rules add a layer on top to suppress notifications during maintenance windows or route them differently.

```
requests
| where timestamp > ago(5m)
| summarize failRate = 100.0 * countif(success == "False") / count()
| where failRate > 5    // fire the log alert above 5% errors
```

GOTCHA

Stateless log alerts can re-fire every evaluation and flood you. Enable stateful alerts and set auto-mitigation so a resolved condition clears, and tune the frequency/threshold so a brief blip does not page the whole team.

86 Azure DevOps services

DEVOPS

Q What are the four core Azure DevOps services and when do you use each?

A zure DevOps is a suite of five (commonly four highlighted) integrated services that cover the software lifecycle end to end. Knowing the split matters because teams often adopt them piecemeal and interconnect them with GitHub or third-party tools.

Boards is agile work tracking: work items, backlogs, sprints, and Kanban boards.

Repos provides Git repositories with branch policies and pull requests. **Pipelines** is the CI/CD engine (build and release), able to deploy anywhere, not just Azure.

Artifacts hosts package feeds (NuGet, npm, Maven, Python) for sharing internal packages. The fifth, **Test Plans**, handles manual and exploratory testing.

The power is the integration: a commit in Repos can close a Boards work item via **#123** in the message, a Pipeline run links back to the work items it delivered, and Artifacts feeds packages into builds. This traceability from requirement to deployment is the reason to keep them together rather than stitching separate tools.

GOTCHA

Microsoft is investing far more in GitHub than in Azure DevOps for new features. For greenfield projects, weigh GitHub (Actions, Advanced Security, Copilot) against Azure DevOps, which remains strong for enterprises needing its granular Boards and on-prem Server option.

87 Azure Pipelines YAML

DEVOPS

Q How is a YAML pipeline structured, and why is YAML preferred over the classic editor?

A YAML pipeline defines your CI/CD as code that lives in the repo alongside the application. This is preferred over the classic UI designer because the pipeline is versioned, reviewable in pull requests, and branchable, so a feature branch can change its own build without affecting main.

The hierarchy is **pipeline** → **stages** → **jobs** → **steps**. A stage is a major phase (build, deploy-to-staging, deploy-to-prod). A job runs on a single agent and is the unit of parallelism. Steps are individual tasks or scripts. Triggers control when it runs, variables and variable groups hold configuration, and templates let you factor out reusable YAML.

For deployments you use a special **deployment job** targeting an Environment, which unlocks approvals, gates, and deployment history, plus strategies like **runOnce**, **rolling**, and **canary**. Connections to Azure use a **service connection**, ideally with workload-identity federation instead of a stored secret.

```
trigger:
  - main
pool:
  vmImage: 'ubuntu-latest'
stages:
  - stage: Build
    jobs:
      - job: build
        steps:
          - script: dotnet build --configuration Release
          - task: PublishBuildArtifacts@1
```

GOTCHA

Microsoft-hosted parallelism for private projects now requires a grant/purchase. Also, variables are strings and boolean-looking values like `true` can surprise you in conditions; wrap comparisons carefully and prefer `eq(variables['x'], 'true')` .

Q How do you deploy to Azure from GitHub Actions without storing secrets?

A he modern approach uses **OpenID Connect (OIDC)** federation instead of a long-lived client secret. The reason this matters: a stored `AZURE_CREDENTIALS` secret is a standing liability that can leak and rarely gets rotated. With OIDC, GitHub mints a short-lived token per run that Azure AD (Entra ID) trusts, so no secret ever lives in the repo.

You configure it by creating an **app registration** (or user-assigned managed identity) in Entra ID and adding a **federated credential** that trusts a specific subject, for example `repo:org/repo:ref:refs/heads/main` or a specific environment. Then you grant that identity RBAC on the target resources. GitHub presents its OIDC token, Azure validates the subject and issuer, and issues an access token.

In the workflow you request `id-token: write` permission and use `azure/login` with client-id, tenant-id, and subscription-id (all non-secret identifiers). No password is passed. Scope the federated subject tightly so only the intended branch or environment can authenticate.

```
permissions:
  id-token: write
  contents: read
steps:
  - uses: azure/login@v2
    with:
      client-id: ${vars.AZURE_CLIENT_ID}
      tenant-id: ${vars.AZURE_TENANT_ID}
      subscription-id: ${vars.AZURE_SUBSCRIPTION_ID}
```

GOTCHA

The federated credential subject must match exactly. Deploying from a tag or a pull_request event fails if your credential only trusts `ref:refs/heads/main` ; add a separate federated credential per subject pattern you need.

Q How do you choose between ARM/Bicep and Terraform on Azure?

Both are declarative infrastructure-as-code tools that provision Azure resources, but they differ in ecosystem, state handling, and multi-cloud reach. Bicep is Microsoft's domain-specific language that transpiles to ARM JSON; Terraform is HashiCorp's cloud-agnostic tool using the AzureRM provider.

The most important operational difference is **state**. Bicep/ARM are stateless from your side: the Azure Resource Manager is the source of truth, and deployments are incremental against live resources, so there is no state file to store or lock. Terraform maintains its own **state file** (which you must store remotely and lock), and it plans by diffing state against reality. State gives Terraform a precise plan but adds operational burden and drift risk if the state and reality diverge.

Choose Bicep when you are Azure-only and want same-day support for new resource types, tight portal/CLI integration, and no state management. Choose Terraform when you are multi-cloud, want a huge module ecosystem, or your org already standardizes on it. Both support what-if/plan previews and modularization.

GOTCHA

ARM incremental mode is the default and never deletes resources not in the template; **Complete** mode does. Terraform, by contrast, will destroy resources you remove from config. Know your tool's deletion semantics before running a deploy in production.

Q What are the key best practices for Infrastructure as Code on Azure?

A IaC treats infrastructure the way you treat application code: versioned, reviewed, and repeatably deployed. The underlying principle is **idempotency**, applying the same definition any number of times yields the same result, which is what makes environments reproducible and disaster recovery credible.

Practically: keep definitions in source control with pull-request review, and parameterize per-environment values rather than forking templates. Use **modules** to encapsulate reusable building blocks (a standard storage account, a hardened VNet). Never hand-edit resources in the portal for IaC-managed infrastructure, because that creates **drift** your next deploy may silently revert or conflict with.

Always run a preview (**what-if** for Bicep, **plan** for Terraform) before applying, and gate it behind CI. Keep secrets out of templates by referencing Key Vault at deploy time. Store Terraform state remotely with locking, tag everything for traceability, and separate stacks by blast radius so a change to one app cannot destabilize shared networking.

```
az deployment group what-if \  
  --resource-group rg-prod \  
  --template-file main.bicep \  
  --parameters env=prod    # preview before you apply
```

GOTCHA

Storing Terraform state in a public or unencrypted location leaks secrets, because state files contain resource attributes in plaintext, including some passwords and keys. Use an encrypted backend with restricted access and never commit **terraform.tfstate** to Git.

91 Azure Policy

GOVERNANCE

Q What is Azure Policy and how does it differ from RBAC?

Azure Policy enforces organizational standards and assesses compliance at scale. The key distinction to internalize: **RBAC controls who can do actions, Policy controls what the resulting resources may look like**. RBAC might let a user create VMs; Policy can insist those VMs use approved SKUs, live in allowed regions, and carry required tags.

A policy definition has an **effect**: **Deny** blocks non-compliant creation, **Audit** flags it without blocking, **Append** / **Modify** adds or changes properties (like tags), **DeployIfNotExists** deploys a remediation (like enabling diagnostics), and **AuditIfNotExists** checks related resources exist. Definitions are grouped into **initiatives** (policy sets) and assigned at a scope: management group, subscription, or resource group.

Policy evaluates both at deploy time (denying bad requests) and continuously (reporting compliance on existing resources). For DeployIfNotExists/Modify effects, the assignment needs a managed identity with rights to remediate, and you run remediation tasks to bring existing resources into line.

GOTCHA

A **Deny** policy does not clean up existing non-compliant resources; it only blocks new or updated ones. To fix what already exists you need a remediation task (DeployIfNotExists/Modify) or manual correction, so audit first to size the gap.

92 Management groups strategy

GOVERNANCE

Q How do management groups fit into an enterprise governance strategy?

A management groups are containers above subscriptions that let you apply governance, RBAC, and Policy across many subscriptions at once. They exist because large orgs have dozens or hundreds of subscriptions, and assigning policy to each individually is unmanageable and error-prone.

They form a hierarchy up to six levels deep under a single **root** management group per tenant. The recommended pattern, from the Cloud Adoption Framework's landing-zone design, groups subscriptions by governance need rather than by org chart: a **Platform** group (identity, management, connectivity subscriptions), a **Landing Zones** group for workloads (often split into corp and online), plus **Sandbox** and **Decommissioned** groups.

Policy and role assignments **inherit** down the tree, so a guardrail set at the intermediate root applies everywhere beneath it. This is powerful and dangerous: a broad Deny at a high level can break every workload below, so you design the hierarchy to match the scope at which each control should apply.

GOTCHA

Avoid organizing management groups to mirror your business/department structure; reorgs then force painful moves. Structure by policy and access requirements instead, and keep the tree shallow so inheritance stays predictable.

93 Cost optimization techniques

GOVERNANCE

Q What are the main levers for optimizing Azure spend?

A cost optimization is a continuous discipline, and the biggest wins come from matching capacity and commitment to real usage. The framing to remember: you pay for **what you provision, not what you use**, so idle and oversized resources are pure waste.

The primary levers are **right-sizing** (downsizing over-provisioned VMs/databases based on utilization metrics), **commitment discounts** (Reserved Instances and Savings Plans trade a 1- or 3-year commitment for up to ~65-72% off), **Azure Hybrid Benefit** (reuse on-prem Windows/SQL licenses), and **Spot VMs** for interruptible batch work at deep discounts. Autoscale and scheduled shutdown of dev/test resources cut the "always-on" tax.

Storage tiering (Hot/Cool/Cold/Archive), deleting orphaned disks and unattached public IPs, and choosing serverless/consumption tiers for spiky workloads round it out. Microsoft Cost Management plus Azure Advisor surface concrete recommendations, and budgets with alerts stop surprises.

GOTCHA

Reservations and Savings Plans are commitments: if you buy a 3-year reservation for a VM series you later abandon, you keep paying. Start with Savings Plans (more flexible across SKUs/regions) for uncertain workloads and reserve only stable, long-lived baselines.

94 Tagging strategy

GOVERNANCE

Q How do you design an effective resource tagging strategy?

A gs are name/value metadata attached to resources that power cost allocation, automation, and governance. A deliberate strategy matters because tags are the primary way to slice a cloud bill by team, environment, or application, and retrofitting them across thousands of resources is painful.

Decide on a small set of **required tags** up front, for example **Environment** , **CostCenter** , **Owner** , **Application** , and **DataClassification** . Standardize casing and allowed values so "Prod" and "prod" do not fragment your reports. Enforce them with Azure Policy: use a **Deny** effect to require a tag, or **Modify** / **Append** to inherit a tag from the resource group automatically.

Remember tags do not inherit by default, a resource does not automatically get its resource group's tags, which is exactly why the inherit-tag policies exist. Keep within limits (50 tags per resource) and treat tags as data your FinOps and automation depend on, not free-form notes.

GOTCHA

Some resource types and older APIs do not support tags, and tags are not automatically inherited by child resources. Cost reports built purely on tags will show untagged spend; pair tagging with resource-group and subscription-level allocation to close the gap.

95 Azure Site Recovery

RESILIENCE

Q What problem does Azure Site Recovery solve, and how is it different from backup?

Azure Site Recovery (ASR) is a disaster-recovery service that continuously **replicates** running VMs to a secondary region (or Azure from on-prem) so you can fail over quickly during an outage. The critical distinction: **backup protects data, DR protects availability**. Backup restores files after loss; ASR keeps a warm, bootable copy of the whole machine ready to run.

ASR replicates disk changes asynchronously and lets you define **recovery plans** that orchestrate failover order, scripts, and manual steps across multi-tier apps. It measures two objectives: **RPO** (how much data you can lose, seconds to minutes with continuous replication) and **RTO** (how fast you can be running again). Crucially, you can run non-disruptive **test failovers** into an isolated network to validate DR without touching production.

Failover can be planned (zero data loss, for maintenance) or unplanned (after a regional disaster). After the primary region recovers, you **reprotect** and fail back. This orchestration and testability is what separates ASR from just copying disks.

GOTCHA

ASR is not a backup: it keeps a limited window of recovery points, so ransomware or corruption can replicate to the secondary. You still need Azure Backup for point-in-time restore. Use both, and test failover regularly, an untested DR plan is a hope, not a plan.

96 Azure Backup strategy

RESILIENCE

Q How do you design a backup strategy with Azure Backup?

Azure Backup provides application-consistent, policy-driven backups for VMs, SQL/SAP in VMs, Azure Files, blobs, and more, stored in a Recovery Services vault or Backup vault. The design goal is to meet each workload's **RPO and retention** requirements while protecting the backups themselves from deletion.

You attach a **backup policy** defining frequency (daily/hourly) and a grandfather-father-son retention scheme (daily, weekly, monthly, yearly points). Match retention to compliance needs, longer retention costs more storage. Choose the right redundancy for the vault: LRS, ZRS, or GRS (with the option to restore cross-region from geo-redundant copies).

The modern must-have is **immutability and soft delete**: soft delete retains deleted backups for a recovery window, and vault immutability plus **multi-user authorization (MUA)** prevents even an admin (or attacker) from destroying recovery points, which is your last line of defense against ransomware.

GOTCHA

The famous rule is 3-2-1: three copies, two media, one off-site. In Azure terms, do not keep your only backups in the same region/subscription as production; use GRS or cross-subscription/cross-region backup, and enable MUA so a compromised admin account cannot wipe them.

97 High availability across zones and regions

RESILIENCE

Q How do you design for high availability using availability zones and regions?

A High availability is about surviving failures without downtime, and Azure gives you a layered set of fault-isolation boundaries to build on. The core idea: distribute across **independent failure domains** so no single hardware, datacenter, or region failure takes the whole system down.

Within a region, **availability zones** are physically separate datacenters with independent power, cooling, and network. Spreading instances across zones (zone-redundant deployments) survives a datacenter loss with low latency between zones. Below that, **availability sets** spread VMs across fault and update domains within one datacenter to survive rack failures and host patching.

For regional disasters you go **multi-region**: pair an active region with a secondary using Azure's paired regions, geo-replicated data (GRS, SQL geo-replication, Cosmos DB multi-region writes), and a global router like Front Door or Traffic Manager to steer traffic. The pattern (active-active vs active-passive) is a cost-vs-RTO tradeoff.

GOTCHA

Zone redundancy protects against a datacenter outage but not a whole-region outage; only multi-region does. And read the SLA fine print: a single VM has a much lower SLA than instances spread across zones, so availability requires design, not just deployment.

98 Microsoft Defender for Cloud

SECURITY

Q What is Microsoft Defender for Cloud and what are its two main pillars?

A Microsoft Defender for Cloud is Azure's cloud-native application protection platform (CNAPP). It answers two questions continuously: "how secure is my posture?" and "am I under attack right now?" Those map to its two pillars, and knowing the split clarifies what is free versus paid.

The first pillar, **Cloud Security Posture Management (CSPM)**, is largely free: it assesses resources against the Microsoft Cloud Security Benchmark, produces a **Secure Score**, and gives prioritized hardening recommendations. It also spans multicloud, connecting AWS and GCP accounts for unified posture.

The second pillar, **Cloud Workload Protection (CWPP)**, is the paid Defender plans that provide threat detection for specific resource types, Defender for Servers (with Microsoft Defender for Endpoint), for Storage (malware scanning), for SQL, for Containers/Kubernetes, and for Key Vault. These generate security alerts that can flow into Microsoft Sentinel for SIEM correlation.

GOTCHA

Foundational CSPM and Secure Score are on by default, but the real threat detection lives in the paid Defender plans, which are off until you enable them per subscription and resource type. A "green" portal with plans disabled gives false comfort, verify each plan's coverage.

99 Migrating with Azure Migrate

MIGRATION

Q How does Azure Migrate support moving workloads to Azure, and what are the migration strategies?

Azure Migrate is the central hub for discovering, assessing, and migrating on-premises servers, databases, web apps, and data to Azure. It matters because a successful migration is driven by data, dependency maps and sizing, not guesswork, and Migrate gathers exactly that before you move anything.

The workflow is **discover, assess, migrate**. A lightweight appliance inventories your VMware/Hyper-V/physical estate, capturing performance data and **dependency visualization** so you group servers that must move together. Assessments then estimate Azure sizing, monthly cost, and readiness, including Azure Hybrid Benefit savings.

Strategy is usually framed as the **5 Rs**: **Rehost** (lift-and-shift, fastest, via server migration replication), **Refactor** (minor changes, e.g., to App Service or containers), **Rearchitect** (redesign for PaaS/cloud-native), **Rebuild** (rewrite), and **Replace** (adopt SaaS). Rehost gets you off-prem quickly; the others deliver more cloud value over time.

GOTCHA

Lift-and-shift without right-sizing just moves inefficiency into Azure at a metered cost. Use assessment data to right-size on the way in, and plan to modernize high-value workloads afterward rather than treating rehost as the finish line.

100 Azure best practices summary

BEST PRACTICES

Q If you had to summarize Azure architectural best practices, what framework and principles would you cite?

A The canonical reference is the Azure Well-Architected Framework, which organizes best practice into five pillars. Citing it signals you design against a shared rubric rather than ad hoc opinions, and it forces you to reason about the tradeoffs between pillars.

The five pillars are **Reliability** (design for failure: redundancy across zones/regions, health probes, retries), **Security** (Zero Trust, least privilege, defense in depth, managed identities over secrets), **Cost Optimization** (right-size, commit for baselines, shut down idle resources), **Operational Excellence** (IaC, CI/CD, monitoring and actionable alerts), and **Performance Efficiency** (scale out, pick the right service tier, cache).

Cross-cutting themes tie it together: prefer **managed/PaaS** services to offload undifferentiated ops, use **managed identities and Key Vault** to eliminate stored credentials, enforce guardrails with Policy and landing zones, and make everything **observable**. The Cloud Adoption Framework complements this at the organizational/landing-zone level.

```
az advisor recommendation list \
  --category Cost Security HighAvailability Performance
OperationalExcellence
# Advisor maps directly onto the WAF pillars
```

GOTCHA

The pillars trade off against each other, maximum reliability and performance raise cost, and heavy security can hurt operational simplicity. "Best practice" is not maximizing every pillar; it is making the tradeoffs explicit for your specific workload and business requirements.

You made it to 100.

If you can explain most of these out loud, you are ready for almost any Azure interview. The people who stand out are not the ones who memorized service names, they are the ones who can justify a choice: why App Service over AKS here, why ZRS over LRS there, and how managed identities remove secrets from the picture entirely. When you hit a question that is not here, walk it the same way: what does this service do, what does it cost, and what runs it more simply?

Found this useful?

This is part of a series of deep-dive interview guides. Follow for the next one, a new technology every few days.

Save this PDF and share it with someone who is interviewing.